## 1. Explain the Internal Architecture of 8086 with a Neat Block Diagram

The **8086 microprocessor architecture** consists of two main functional units:

**Bus Interface Unit (BIU)**

Responsible for:

- Fetching instructions from memory
- Handling addresses and data transfers
- Managing the **Instruction Queue (6 bytes)** to implement **pipelining**
- Address calculation using segment and offset addresses

**Execution Unit (EU)**

Responsible for:

- Decoding and executing instructions
- Contains **ALU**, **Control unit**, and **Registers**
- Sends operand requests to BIU
- Executes arithmetic and logic operations

**Key Features:**

- **Pipelining**: BIU fetches next instructions while EU executes current instruction
- **Registers**: 14 general and special-purpose 16-bit registers
- **Memory Segmentation**: 4 segment registers allow access to 1MB of memory

**Block diagram** includes:

- ALU, Control unit, General registers (AX, BX, CX, DX), Segment registers (CS, DS, SS, ES), Instruction Pointer, and Instruction Queue in BIU.

## 2. Explain Briefly Any 4 Addressing Modes of 8086 with Examples

Addressing modes define **how operands are accessed** in instructions. Four important ones are:

**Immediate Addressing**

Operand is part of instruction.

- Example: `MOV AX, 1234H`
  (1234H is immediate data)

**Register Addressing**

Operands are in registers.

- Example: `MOV AX, BX`

**Direct Addressing**

Operand address is given explicitly.

- Example: `MOV AX, [5000H]`
  (Accesses memory location 5000H)

**Register Indirect Addressing**

Address is stored in a register.

- Example: `MOV AX, [BX]`
  (Memory address is in BX)

Other modes include **Base + Index**, **Relative**, and **Indexed addressing** for flexible memory access.

## 3. Explain Briefly Any 4 Assembler Directives

Assembler directives help **organize and define structure** of programs. They are not executable instructions.

**ORG (Origin):**

- Sets starting address for code/data.
- Example: `ORG 2000H`

**DB (Define Byte):**

- Used to define 8-bit data.
- Example: `DATA DB 55H`

**EQU (Equate):**

- Defines constants.
- Example: `PI EQU 3.14`

**END:**

- Marks end of source code.
- Example: `END`
  Everything after it is ignored during assembly.

## 4. Explain the Following 8086 Instructions

| Mnemonic | Description |
|---|---|
| **XLAT** | Translate byte in AL using a lookup table pointed by BX. Useful in table-based conversion. |
| **NEG** | Negates operand. 2's complement operation. `NEG AL` → AL = –AL |
| **LEA** | Load Effective Address. Example: `LEA SI, [BX+DI]` loads address, not data. |
| **IDIV** | Signed division. Result → quotient in AX, remainder in DX |
| **MOVSB** | Move byte from [SI] to [DI]. Used for string operations |
| **XOR** | Logical Exclusive-OR. Often used to clear registers (`XOR AX, AX`) |
| **STD** | Set Direction Flag → String ops decrement addresses |
| **RCR** | Rotate bits right through carry flag. Used in bit-level manipulations. |

## 5. Write the General Bus Operation of 8086 Microprocessor

**Overview:**

8086 performs bus operations to transfer **data and addresses** between the microprocessor and memory or I/O devices.

### Bus Operations Include:

- **Opcode Fetch**: Fetch instruction from memory.
- **Memory Read/Write**: Read/write data from/to memory.
- **I/O Read/Write**: Communicate with external devices.
- **Interrupt Acknowledge**: Acknowledge interrupt requests.
- **DMA Acknowledge**: Support Direct Memory Access.

### Control Signals Involved:

- **ALE (Address Latch Enable)**: Distinguishes between address/data on multiplexed lines.
- **RD' / WR'**: Read or write signal.
- **M/IO'**: Specifies memory or I/O operation.
- **DT/R', DEN'**: Direction and enable data buffer.
- **BHE'/A0**: Used to access even/odd bytes or both simultaneously

## 6. Explain the Difference Between RISC and CISC Architecture

**RISC (Reduced Instruction Set Computer)**

- Uses a **small and simple instruction set**.
- Focuses on **instructions that execute in one clock cycle**.
- Requires **more instructions** to accomplish complex tasks.
- Example: **Microchip PIC18F87X**

**CISC (Complex Instruction Set Computer)**

- Supports a **large set of complex instructions**.
- Instructions may take **multiple cycles** to execute.
- Easier for **programmers**, as fewer instructions are needed.
- Example: **Intel 8051**

**Comparison Table:**

| Feature | RISC | CISC |
|---|---|---|
| Instruction Size | Fixed, small | Variable |
| Execution Speed | Fast (1 instruction per cycle) | Slower (multi-cycle) |
| Hardware Usage | Simpler hardware | Complex hardware |
| Examples | PIC microcontrollers | 8051, Intel x86 |

## 7. What Are Assembler Directives? Explain the Following

**Assembler Directives are instructions to the assembler, not the CPU. They help organize code and data.**

**i) ASSUME**

Tells the assembler which segment registers should be used for code or data.
Example: `ASSUME CS:CODE, DS:DATA`
Meaning: Instructions will use CODE segment, data will use DATA segment.

**ii) DUP (Duplicate)**

Used to **initialize a block of memory**.
Example: `ARR DB 5 DUP(0)`
Means: Reserve 5 bytes of memory, each initialized to 0.

**iii) DB (Define Byte)**

Defines **byte-sized data**. Most commonly used directive.
Example: `NAME DB 'A'` → Stores ASCII for 'A'
Can define binary, decimal, or hex values

**iv) LABEL**

Used to give a **symbolic name** to an instruction or data location.
Example: `START: MOV AX, 0` Here, `START` is a label that can be used for jumps or reference.

## 8. Explain the Flag Register of 8086 in Detail with a Neat Diagram

The **8086 flag register** is a **16-bit register** used to indicate results and control execution flow.

**Divided Into:**

- **Status Flags (Lower byte)**
- **Control Flags (Upper byte)**

### Status/Condition Flags:

| Flag | Description |
|---|---|
| **CF** (Carry) | Set if there is a carry out/borrow in arithmetic ops |
| **PF** (Parity) | Set if result has even parity |
| **AF** (Auxiliary Carry) | Carry between lower and upper nibbles |
| **ZF** (Zero) | Set if result = 0 |
| **SF** (Sign) | Set if result is negative |
| **OF** (Overflow) | Set if signed overflow occurs |

### Control Flags:

| Flag | Description |
|---|---|
| **TF** (Trap) | Enables single-step execution |
| **IF** (Interrupt) | Enables/disables maskable interrupts |
| **DF** (Direction) | Controls string instruction direction |

Diagram is not in the document, but a neat layout can be:

```
15    14 13 12 11 10  9  8   7  6  5  4  3  2  1  0
 -     -  -  -  OF DF IF TF  SF ZF - AF - PF - CF
```

## 9. Explain the String Instructions of 8086

These instructions operate on **strings of bytes or words**. They use **SI (source index)** and **DI (destination index)** registers.

**Common String Instructions:**

| Mnemonic | Meaning | Description |
|---|---|---|
| **MOVSB / MOVSW** | Move byte/word | `[SI] → [DI]` |
| **LODSB / LODSW** | Load accumulator | `[SI] → AL/AX` |
| **STOSB / STOSW** | Store accumulator | `AL/AX → [DI]` |
| **CMPSB / CMPSW** | Compare strings | Compares `[SI]` and `[DI]` |
| **SCASB / SCASW** | Scan string | Compares `AL/AX` to `[DI]` |

**Used with REP Prefix:**

- **REP**: Repeat until CX = 0
- **REPE/REPZ**: Repeat while equal/zero
- **REPNE/REPNZ**: Repeat while not equal/non-zero

**Direction Controlled by DF Flag:**

- **DF = 0** → Auto-increment
- **DF = 1** → Auto-decrement

## 10. Sketch the Minimum Mode Operation of 8086 and Explain Briefly

**Minimum Mode:**

Activated by setting **MN/MX' pin = 1**. Used when **only one processor** is active.

### Components in Minimum Mode System:

- 8086 Microprocessor
- Clock Generator
- Latches (74LS373)
- Transceivers (74LS245)
- Memory (RAM/ROM)
- I/O Devices

### Key Signals Used:

| Signal | Description |
|---|---|
| **ALE** | Address Latch Enable (used with latches) |
| **M/IO'** | Memory or I/O access |
| **RD'/WR'** | Read/Write control |
| **DT/R'** | Direction of data |
| **DEN'** | Data enable |
| **INTA'** | Interrupt Acknowledge |

### Operation Summary:

- **ALE** signals that address is valid → used to latch address.
- **M/IO'**, **RD'**, **WR'** used to control memory or I/O reads/writes.
- **Transceivers** manage data direction using **DT/R'** and **DEN'**.

### Timing Diagrams:

- **T1–T4 states** define memory read/write timing.

- **Opcode fetch** and **memory access** have similar timing.
- Separate **read** and **write cycle diagrams** illustrate control signal transitions.

## 11. Sketch the Maximum Mode Operation of 8086 and Explain Briefly

**Maximum Mode Overview:**

8086 enters **maximum mode** when the **MN/MX' pin is set to 0** (connected to GND). This mode supports **multiprocessor configurations** where the control signals are **not directly generated** by the microprocessor.

Instead, **status signals (S2, S1, S0)** are generated and a **bus controller (Intel 8288)** derives the necessary control signals.

**Functional Flow:**

- **Status Lines (S2, S1, S0)** provide the type of operation (e.g., Memory Read, I/O Write, etc.).
- **Bus Controller (8288)** decodes the status signals and generates:
    - **MRDC' (Memory Read Command)**
    - **MWTC' (Memory Write Command)**
    - **IORC', IOWC' (I/O Read/Write Commands)**
    - **DEN (Data Enable)**, **DT/R (Direction)**, **INTA (Interrupt Acknowledge)**, etc.

**Timing and Signals:**

- **ALE**: Address Latch Enable
- **DT/R'**: Data Transmit/Receive
- **DEN'**: Data Enable
- **INTA'**: Two-pulse interrupt acknowledgment
- **MRDC', MWTC'**: Memory control signals
- **IORC', IOWC'**: I/O control signals

**Important Notes:**

- Control logic is **offloaded to 8288**, improving efficiency in multiprocessor systems.
- Read/write timings and signal interpretations differ from **minimum mode**.

**Diagram and signal tables** are provided in the document, showing connection between CPU, 8288 bus controller, memory, and I/O.

## 12. Explain the Physical Address Formation in 8086 with an Example

**Key Concepts:**

8086 has:

- **16-bit segment registers**: CS, DS, SS, ES
- **16-bit offset**: IP, SI, DI, BP, SP, etc.
- **Physical address** is 20-bit.

**Formula:**

**Physical Address = (Segment Register × 10H) + Offset**

**Example:**

Let:

- DS = 2450H
- SI = 4321H

Then,
**Physical Address = (2450H × 10H) + 4321H = 24500H + 4321H = 28821H**

This address points to the memory location accessed.

Memory segmentation allows **overlapping** and **efficient memory use** in a 1MB address space.

## 13. Differentiate Between the Following Instructions

### i) AND vs TEST

| Feature | AND | TEST |
|---|---|---|
| Operation | Logical AND + stores result | Logical AND (no result saved) |
| Flags affected | All logic flags | Same flags, no data change |
| Usage | Modify destination bits | Check bit pattern |
| Example | `AND AL, BL` | `TEST AL, 01H` |

### ii) SHIFT vs ROTATE

| Feature | SHIFT | ROTATE |
|---|---|---|
| Bit Movement | Moves bits left/right | Rotates bits with or without carry |
| Carry Use | Optional | May involve Carry Flag (RCL/RCR) |
| Data Loss | Yes | No (bits re-enter) |
| Example | `SHL AX, 1` | `ROL AX, 1` |

ROTATE instructions include:

- `RCL`, `RCR` (Rotate through Carry)
- `ROL`, `ROR` (Rotate Left/Right)

## 14. Briefly Explain the Following 8086 Instructions

**i) `MOV AX, [SI]`**

- Moves word from memory at address pointed by SI into AX.
- Assumes DS is default segment.
- **Effective Address**: `[DS:SI]`

**ii) `ADD BYTE PTR [DI], 3`**

- Adds 3 to the byte at memory location pointed by DI.
- DI is offset; DS is default segment.

**iii) `MOV AL, [BX]`**

- Moves byte from memory at offset BX into AL.
- Uses DS:BX as memory address.

**iv)** `MOV 56[SI], CL`

- Stores the contents of CL into the memory location offset by 56 from SI.
- Actual Address = `[DS:SI+56]`

Each instruction demonstrates different **addressing modes**: register indirect, base-displacement, and indexed addressing.

## 15. Explain the Significance of the Following Pins of 8086

### i) ALE (Address Latch Enable)

- Indicates the presence of a valid address on the multiplexed address/data lines (AD0–AD15).
- Used to **latch address** before data is placed on the same lines.
- **Rising edge** triggers latch.

### ii) RESET

- Resets the processor and restarts execution from address `FFFF0H`.
- Must be **high for at least 4 clock cycles**.
- Sets IP = 0000H and CS = FFFFH.

### iii) TEST

- Input signal tested by the `WAIT` instruction.
- If TEST = low → instruction proceeds.
- If TEST = high → CPU waits.
- Used in **coprocessor synchronization**.

### iv) M/IO (Memory/Input-Output)

- Selects whether the operation is **memory** or **I/O**.
- High = memory operation.
- Low = I/O operation.
- Used in **minimum mode** only.

All of the above are part of the 8086's rich pin configuration supporting **multi-functional bus usage and synchronization mechanisms**.

## 16. Physical Address Calculation

**Formula: Physical Address = (Segment × 10H) + Offset**

Given:

- **CS = 1000H**, **DS = 25A0H**, **SS = 3210H**, **ES = 5890H**
- **BX = 43A9H**, **BP = 3400H**

**i)** `MOV AL, [BX+1200]`

This uses DS as segment by default.

- Offset = BX + 1200H = 43A9H + 1200H = **55A9H**
- DS = 25A0H → DS × 10H = 25A00H

- **Physical Address = 25A00H + 55A9H = 2AFA9H**

**ii)** `ADD BX, [BP+05]`

This is a memory operand using **BP**, so default segment = **SS**

- Offset = BP + 05 = 3400H + 05 = **3405H**
- SS × 10H = 32100H
- **Physical Address = 32100H + 3405H = 35505H**

## 17. Instruction Format for 8086 (ADD)

**i) Register to Register:** `ADD AX, BX`

| Field | Value |
|---|---|
| Opcode | 03 (for ADD r16, r/m16) |
| MOD | 11 (register mode) |
| REG | 000 (AX) |
| R/M | 011 (BX) |
| Binary | 03 C3 |

**Opcode = 03C3**

**ii) Immediate to Register:** `ADD CX, 1200H`

| Field | Value |
|---|---|
| Opcode | 81 (ADD r/m16, imm16) |
| MOD | 11 |
| REG | 000 |
| R/M | 001 (CX) |
| Immediate | 1200H (low byte first) |
| Opcode | 81 C1 00 12 |

**Opcode = 81C10012**

## 18. Instruction Check and Final Flag Result

**Instructions:**

I. `MOV BX, 0804H` → OK
II. `INC [BX+02]` → Valid memory increment
III. `ADD 06H, AL` → ✕ Invalid syntax (Correct: `ADD AL, 06H`)
IV. `SHR DX, 02` → OK (logical shift right)
V. `XOR BL, BL` → Sets BL to 0, and flags (Z=1, P=1, S=0, CY=0)

## Result After Execution (assuming initial flags reset):

- **BX = 0804H**
- **Flags:**
  - **CY = 0** (No carry)
  - **Z = 1** (BL = 0)
  - **S = 0** (Positive)

- **P = 1** (Even parity)

## 19. Describe the Following Signals of 8086

**i) MN/MX̄**

- Selects **Minimum or Maximum Mode**
- `1` → Minimum mode
- `0` → Maximum mode
- Affects pin functionality (e.g., whether 8288 is needed)

**ii) ALE (Address Latch Enable)**

- Used to latch address from multiplexed **AD0–AD15**
- **Rising edge** indicates address is valid

**iii) BHE̅ / S7**

- **BHE̅ (Bus High Enable)**: Enables data bus D8–D15
- Used for **odd-address byte access**
- **S7** is a status bit in maximum mode

**iv) RD̅ / GT̅C**

- `RD̅`: Low when reading from memory/I/O
- `GT̅C`: Request/Grant control used for **bus arbitration** in **maximum mode**

## 20. Addressing Modes of 8086 with Explanation

**i) Register Addressing Mode**

- Operand is in a register.
- Example: `MOV AX, BX`

**ii) Based Indexed Addressing Mode**

- Combines base register (BX/BP) and index (SI/DI).
- Example: `MOV AX, [BX+SI]`

**iii) Immediate Addressing Mode**

- Operand is part of instruction.
- Example: `MOV AL, 34H`

**iv) Direct Addressing Mode**

- Operand is in memory, and full address is given.
- Example: `MOV AX, [1234H]`

## 19. Describe the Following Signals of 8086 Microprocessor

The **8086 microprocessor** has multiple **control and status signals**. These signals coordinate memory access, I/O operations, and data transfer. Below is a detailed explanation of the listed signals:

## i) MN/M$\overline{X}$ (Minimum/Maximum Mode Select Pin)

- This **active-low pin** determines the **operating mode** of the 8086:
  - **MN/M$\overline{X}$ = 1**: **Minimum Mode**
    - Used in **single processor** systems.
    - 8086 itself generates all control signals like **R$\overline{D}$, W$\overline{R}$, INT$\overline{A}$, DEN, ALE**, etc.
  - **MN/M$\overline{X}$ = 0**: **Maximum Mode**
    - Used in **multiprocessor** environments.
    - External **bus controller (Intel 8288)** is needed to generate control signals.
    - 8086 outputs **status signals (S0, S1, S2)** instead of control signals.

**Importance:**

- Enables flexibility in design: single processor (min) or multiprocessor (max) configuration.
- Mode affects pin functions and timing behavior.

## ii) ALE (Address Latch Enable)

- ALE is an **output signal** used to **demultiplex** address/data lines **AD0–AD15**.
- In the 8086, address and data share the same lines.
- **Rising edge of ALE** indicates:
  - Valid **address** is on the AD lines.
  - Latches (like 74LS373) should capture the address.

**Used to:**

- Separate **address phase** from **data phase** in memory and I/O operations.
- Allow external devices to store the address temporarily.

## iii) BH$\overline{E}$ / S7 (Bus High Enable / Status Bit 7)

### a) BH$\overline{E}$ (Bus High Enable):

- **Active-low signal** indicating a valid transfer on the **higher byte** (D8–D15) of the data bus.
- Used in conjunction with **A0** to select the appropriate memory byte(s).

| BH$\overline{E}$ | A0 | Byte Accessed |
|---|---|---|
| 0 | 0 | 16-bit (both bytes) |
| 0 | 1 | High byte only |
| 1 | 0 | Low byte only |
| 1 | 1 | Invalid |

### b) S7 (in Max Mode):

- When in maximum mode, this pin outputs **status line S7**.
- Reserved for future use; not functionally active in 8086.

## iv) R$\overline{D}$ / G$\overline{T}$C (Read / Grant Acknowledge Control)

### a) R$\overline{D}$ (Read Control):

- **Active-low output signal**.
- Generated when 8086 **reads** data from memory or I/O.
- Used to **enable the memory or I/O device** to put data on the bus.

**b) G$\overline{T}$C (Grant Control):**

- In **maximum mode**, this pin is used for **bus arbitration**.
- Works with **R$\overline{Q}$/G$\overline{T}$ (Request/Grant)** signals.
- Indicates that the bus has been granted to another master.
- Used in **multiprocessor** or **DMA (Direct Memory Access)** systems.

## Summary Table

| Signal | Function in 8086 |
|---|---|
| **MN/M$\overline{X}$** | Selects minimum (1) or maximum (0) mode |
| **ALE** | Enables latching of address (AD0–AD15) |
| **BH$\overline{E}$/S7** | Selects upper byte of data (BH$\overline{E}$); status bit S7 in max mode |
| **R$\overline{D}$/G$\overline{T}$C** | Controls read operation; grant bus access in max mode |

## 20. Explain the Addressing Modes of 8086

The **8086 microprocessor** supports a variety of **addressing modes** that define how operands are accessed during execution of instructions. These allow for flexible data access from registers, memory, or immediate values.

### i) Register Addressing Mode

- Operands are **residing in registers**.
- Fastest mode because no memory access is involved.

**Syntax:** `MOV AX, BX`
**Explanation:** Contents of **BX** are moved to **AX**.
Both operands are general-purpose 16-bit registers.

### ii) Based Indexed Addressing Mode

- Combines **base register** and **index register** to calculate the effective address.

**Registers Used:**

- Base: **BX** or **BP**
- Index: **SI** or **DI**

**Syntax:** `MOV AX, [BX+SI]`
**Explanation:**

- Effective address = BX + SI
- Data at that memory location is moved into AX.
- **DS** is used as default segment if BX used; **SS** if BP used.

### iii) Immediate Addressing Mode

- Operand is a **constant** directly embedded in the instruction.

**Syntax:** `MOV AL, 34H`
**Explanation:**

- 34H is loaded into register AL.
- No memory access required; data is part of instruction itself.

### iv) Direct Addressing Mode

- Instruction specifies the **full memory address** of the operand.

**Syntax:** `MOV AX, [1234H]`
**Explanation:**

- Loads the word from memory location 1234H (in segment **DS**) into AX.
- Effective address = DS × 10H + 1234H

### Additional Addressing Modes for Depth (Bonus for 15 Marks):

| Mode | Example | Description |
|---|---|---|
| Register Indirect | `MOV AX, [BX]` | Address held in a register |
| Relative | `JMP SHORT LABEL` | Jump relative to IP |
| Indexed | `MOV AL, [SI]` | Uses SI or DI as offset |
| Displacement | `MOV AX, [BX+10H]` | Combines base register and offset |

### Why Addressing Modes Matter:

- They provide **flexibility and efficiency** in accessing operands.
- Allow access to **arrays, pointers, constants, and memory structures** easily.

**Module2 1.** Explain the Stack Structure of 8086 and the Operations of PUSH and POP Instructions

### What is a Stack?

- A **stack** is a **Last In First Out (LIFO)** data structure used to store temporary data.
- In 8086, the stack is **memory-based**, and it grows **downward** (i.e., from high memory to low memory).

### Registers Involved:

- **SP (Stack Pointer)**: 16-bit register holds offset address within stack segment.
- **SS (Stack Segment)**: Holds base segment address of the stack.

**Physical address of top of stack:**

Physical Address=SS×10H+SP

### PUSH Instruction:

- Stores a 16-bit register or memory word onto the stack.
- Decrements SP by 2 (since 8086 is 16-bit).
- Stores high byte first, then low byte.

**Example:** `PUSH BX`

- SP = SP – 2
- Memory[SS:SP] = BX

### POP Instruction:

- Retrieves (pops) the last 16-bit value from the stack into a register.
- Reads data from [SS:SP], then increments SP by 2.

**Example:** `POP AX`

- AX = Memory[SS:SP]
- SP = SP + 2

### Applications of Stack:

- Saving return addresses during subroutine calls
- Context switching during interrupts
- Temporary data storage

2. **Sketch the content of stack memory indicating the value of SP register before PUSH BX operation and after the PUSH BX operation. Assume SS=2500 H, BX=432AH and SP=1000 H.**

### Given:

- `SS = 2500H`
- `SP = 1000H`
- `BX = 432AH`

### Before PUSH BX:

- Top of stack: Physical Address = `25000H + 1000H = 26000H`
- SP = 1000H
- Stack is empty at this point

### PUSH BX Operation:

1. SP = SP - 1 → 0FFFH → store **high byte** of BX = 43H
2. SP = SP - 1 → 0FFEH → store **low byte** of BX = 2AH

### After PUSH BX:

- SP = 0FFEH
- Stack content:

| Address | Value | Description |
|---|---|---|
| 25FFFH | 43H | High byte of BX |
| 25FFEH | 2AH | Low byte of BX |

### Memory View:

```
SP: 0FFEH → 2A
     0FFFH → 43
```

BX is now stored at top of stack.

### 3. What are the sequence of actions taken by 8086 and the device, when a device interrupts 8086 over INTR line? Explain about the software and reserved interrupts of 8086

### INTR (Interrupt Request)

INTR is a **maskable hardware interrupt**, triggered by external devices to request service.

### Sequence of Actions (Interrupt Cycle):

1. **INTR Line is activated** by a device.
2. If **IF (Interrupt Flag) = 1**, 8086:
    - Finishes current instruction.
    - Sends **INTA (Interrupt Acknowledge)**.
3. **External device** places **interrupt type number** (0–255) on the data bus.
4. 8086:
    - Clears IF and TF (disables further interrupts).
    - Pushes FLAGS, CS, and IP onto stack.
    - Loads new CS:IP from **Interrupt Vector Table (IVT)**.

### Software Interrupts:

- Triggered using `INT n` instruction.
- Example: `INT 21H` (used in DOS)
- Internally same as hardware interrupt, but triggered by program.

### Reserved Interrupts:

| Type | Description |
|---|---|
| 0 | Divide-by-zero |
| 1 | Single-step (trap) |
| 2 | Non-maskable interrupt |
| 3 | Breakpoint interrupt |
| 4 | Overflow (INTO) |

These have **fixed vector addresses** and are **internally generated** by the processor.

## 4. Explain Interrupt Response Structure of 8086

### Interrupt Response Flow:

When an interrupt occurs (hardware or software), 8086 performs the following sequence:

1. **Checks flags**:
   - If INTR: IF must be 1.
   - If NMI: Always responded.
2. **Finishes current instruction**
3. **Pushes onto stack**:
   - FLAGS register (2 bytes)
   - CS register (2 bytes)
   - IP register (2 bytes)
4. **Clears** IF and TF (disables further interrupts)
5. **Fetches new IP and CS** from Interrupt Vector Table:
   - Address = Type × 4
   - IP from [Type × 4], CS from [Type × 4 + 2]
6. **Jumps to ISR (Interrupt Service Routine)**
7. **ISR Executes**
8. Ends with `IRET` (Interrupt Return):
   - Pops IP, CS, FLAGS in reverse order
   - Resumes interrupted program

### Diagram (descriptive view):

```
[ INTR or INT n ] →
[ Save IP, CS, FLAGS ] →
[ Load new IP, CS from IVT ] →
[ Execute ISR ] →
[ IRET ] →
[ Resume normal execution ]
```

### Types of Interrupts (for Summary):

| Type | Triggered By | Vector Location |
|---|---|---|
| Hardware | INTR, NMI | From IVT |
| Software | INT n | From IVT |
| Internal | Errors (e.g., divide by 0) | Predefined types |

## 5. Describe the purpose of the interrupt vector table of 8086 processor and conditions which cause the following interrupts Type0; Type 2; Type 4;

### Interrupt Vector Table (IVT):

- The **Interrupt Vector Table** is a **reserved memory area** starting at **00000H**.
- It stores **interrupt vectors** — i.e., the **starting address (CS:IP)** of each **Interrupt Service Routine (ISR)**.
- Each interrupt type has a **4-byte entry**:
  - 2 bytes → **IP (Offset)**
  - 2 bytes → **CS (Segment)**
- Since there are 256 interrupt types, IVT requires:

  256 interrupts×4 bytes=1024 bytes (00000H to 003FFH)256 \text{ interrupts} \times 4 \text{ bytes} = 1024 \text{ bytes (00000H to 003FFH)}

## Conditions Causing Specific Interrupts:

| Interrupt | Type No. | Cause | Vector Address |
|---|---|---|---|
| **Divide-by-zero** | Type 0 | DIV or IDIV with zero divisor or overflow | 0000H |
| **NMI (Non-Maskable Interrupt)** | Type 2 | Edge-triggered hardware interrupt (pin-based) | 0008H |
| **Overflow Interrupt** | Type 4 | Software-triggered via `INTO` when Overflow Flag (OF=1) | 0010H |

## Use of IVT:

- Allows **predefined locations** for interrupt routines.
- Enables both **hardware and software interrupts** to use consistent entry points.
- Facilitates **modular and vectored** interrupt handling.

6. Explain the type of interrupts and the action taken by the 8086 when an interrupt occurs in detail.

## Types of Interrupts in 8086:

| Type | Description | Triggered By |
|---|---|---|
| **Hardware Interrupts** | Via pins (INTR or NMI) | External devices |
| **Software Interrupts** | Using `INT n` instruction | Code/Program logic |
| **Internal (Exception)** | Caused by execution errors like divide-by-zero | CPU generated |

## Actions Taken When an Interrupt Occurs:

1. **Complete current instruction**
2. **Clear TF and IF** (disables other maskable interrupts temporarily)
3. **Push current FLAGS, CS, IP** onto the stack
4. **Fetch ISR Address from IVT**
   - IP = [Type × 4]
   - CS = [Type × 4 + 2]
5. **Jump to ISR (Interrupt Service Routine)**
6. ISR executes and ends with `IRET`
7. **POP IP, CS, FLAGS** and continue interrupted program

## NMI vs INTR:

| Feature | NMI | INTR |
|---|---|---|
| Type | Non-maskable | Maskable |
| Priority | High | Lower |
| Pin | Dedicated NMI pin | Shared INTR pin |
| Acknowledgement | No | Requires INTA cycle |

## Software Interrupt Example:

```
INT 21H ; Call DOS interrupt services
```

Triggers type 21H interrupt, which executes an OS-provided ISR.

## 7. Explain the interrupt acknowledgement cycle of 8086 with the neat timing diagram.

### Applicable Only for INTR Interrupt

8086 uses a **2-clock interrupt acknowledge cycle** to service **INTR** requests.

### Sequence:

1. **INTR line goes HIGH**
2. 8086 checks after current instruction and:
   - If **IF = 1**, begins **acknowledge cycle**
3. 8086 **sends INTA' (active-low)** signal **twice**:
   - First INTA': acknowledges request
   - Second INTA': device puts **interrupt type number (0–255)** on **data bus**
4. 8086 reads the type number and fetches ISR from IVT.

### Timing Diagram (Described):

```
INTR: ──────────┐──────────
                │
CLK:  ────┐────┐────┐────┐────┐
          │    │    │    │    │
INTA':    └────┘    └────┘
Data Bus:      ← ← ← Interrupt Type (during 2nd INTA')
```

- During **first INTA' pulse**, device prepares to place type on bus.
- During **second INTA' pulse**, device outputs the type number, which is latched by 8086.

### Note:

- This mechanism ensures proper **handshaking** and **vector assignment** in **multiplexed interrupt environments**.

## 8. Explain the various use cases of stack with a suitable example.

The **stack in 8086** is essential for managing data flow, especially during subroutines and interrupts.

### Key Use Cases:

#### Subroutine Calls (`CALL` and `RET`)

- Stack stores **return address** when a function is called.

```
CALL SUBROUTINE
...
SUBROUTINE:
  ; some code
  RET
```

- On `CALL`, CS and IP are **pushed**
- On `RET`, they are **popped**

**Interrupt Handling**

- When interrupt occurs, stack stores:
  - **FLAGS**
  - **CS**
  - **IP**
- After `IRET`, processor resumes from original location.

**Temporary Register Storage (PUSH/POP)**

```
PUSH AX
...
; modify AX
POP AX  ; Restore original AX
```

Used to save values across critical code sections or function calls.

**Recursive Function Support**

Each function call pushes return address and parameters to the stack.

**Parameter Passing to Procedures**

- Parameters can be passed via **stack**, allowing multiple functions to use the same code logic.

## Stack Growth Direction:

- **Downward** (from high address to low address).
- Each `PUSH` → SP decreases by 2.
- Each `POP` → SP increases by 2.

## Stack Memory View (Example)

| SP Value | Content |
|---|---|
| 0FFEH | Lower byte (e.g., AL) |
| 0FFFH | Higher byte (e.g., AH) |

## Module3 1. What is a Microcontroller? Differences between RISC and CISC

## Definition: Microcontroller

A **microcontroller** is a compact integrated circuit designed to perform a specific operation in an embedded system. It contains:

- **CPU (processor)**
- **Memory (RAM + ROM)**
- **I/O ports**
- **Timers, counters, serial interface**
- **All on a single chip**

Used in appliances, automobiles, robotics, medical devices, etc.

## RISC vs CISC – Key Differences

| Feature | RISC (Reduced Instruction Set Computer) | CISC (Complex Instruction Set Computer) |
|---|---|---|
| **Instruction Set** | Small, optimized, fixed-length instructions | Large set, complex multi-cycle instructions |
| **Execution Speed** | Fast – single cycle per instruction | Slower – multi-cycle execution |
| **Memory Access** | Load/store architecture | Direct memory operations allowed |
| **Hardware Complexity** | Low (more software control) | High (instruction decoder complexity) |
| **Code Size** | Larger (more instructions needed) | Smaller (fewer instructions) |
| **Example Devices** | ARM Cortex, PIC microcontrollers | Intel 8051, x86 processors |

**In 8051 architecture**, it is considered **CISC**, while modern embedded MCUs (like ARM) are **RISC**.

2.Explain the 8051 Block Diagram and Its Features

## Block Diagram Overview of 8051:

8051 microcontroller consists of the following functional blocks:

- **CPU**: 8-bit processor
- **Registers**: R0–R7, accumulator (A), B, PC, SP, PSW
- **Memory**:
  - 128 bytes of internal RAM
  - 4 KB of internal ROM
- **Timers**: Two 16-bit timers (Timer 0 and Timer 1)
- **Serial Port**: UART for serial communication
- **I/O Ports**: 4 ports (P0 to P3), 8-bit each
- **Interrupts**: 5 interrupt sources (2 external, 3 internal)

## Key Features of 8051:

| Feature | Details |
|---|---|
| **Architecture** | Harvard (separate code & data memory) |
| **Data Bus Width** | 8-bit |
| **Program Memory** | 4 KB ROM |
| **Data Memory** | 128 Bytes RAM |
| **I/O Ports** | 4 ports: P0–P3 (8-bit each) |
| **Timers** | 2 × 16-bit timers/counters |
| **Serial Port** | Full duplex UART |
| **Interrupts** | 5 (INT0, T0, INT1, T1, Serial) |
| **Oscillator** | 12 MHz crystal, 1 µs cycle time |

The CPU communicates with peripherals and memory via **internal buses**, ALU, and registers.

## 3.Difference Between Microprocessor and Microcontroller

| Feature | Microprocessor | Microcontroller |
|---|---|---|
| **Definition** | CPU on a chip | CPU + memory + I/O + peripherals on one chip |
| **Memory** | External RAM and ROM needed | Internal RAM/ROM |
| **Application** | General-purpose (PCs, servers) | Specific tasks (embedded systems) |
| **I/O Ports** | Few or none; require external chips | Multiple built-in I/O ports |
| **Timers/Counters** | External peripherals required | Built-in |
| **Size & Cost** | Larger, more expensive | Smaller, low cost |
| **Examples** | Intel 8086, Pentium, AMD Ryzen | 8051, AVR, PIC, ARM Cortex |

**Microcontroller = Microprocessor + RAM + ROM + I/O + Timers on a single chip**

## 4.Describe the Functions of Various Pins of 8051 with Pin Diagram

### 8051 has 40 pins, grouped as:

- Power pins
- Oscillator pins
- Reset
- I/O ports
- Control & status
- Serial communication

### Pin Diagram Overview:

```
   ┌──────────┐
P1.0 | 1   40 | Vcc
P1.1 | 2   39 | P0.0 / AD0
...   |       |
RST  | 9   32 | P3.7 / RD
EA   | 31  10 | P3.0 / RXD
...   |       |
GND  | 20  21 | P2.0
   └──────────┘
```

### Important Pin Functions:

| Pin Name | Pin No | Description |
|---|---|---|
| **Vcc** | 40 | +5V supply |
| **GND** | 20 | Ground |
| **XTAL1, XTAL2** | 18, 19 | External oscillator crystal (e.g., 12 MHz) |
| **RST** | 9 | Reset input (high for 2 cycles to reset) |
| **EA** | 31 | External Access (0 = external ROM, 1 = internal) |
| **PSEN** | 29 | Program Store Enable (used to read external ROM) |
| **ALE** | 30 | Address Latch Enable for AD0–AD7 |

| Pin Name | Pin No | Description |
|---|---|---|
| **Port 0** | 32–39 | AD0–AD7 (multiplexed address/data) |
| **Port 1** | 1–8 | General I/O (bidirectional) |
| **Port 2** | 21–28 | High address bus (for external memory) |
| **Port 3** | 10–17 | Alternate functions (INT, T0/T1, TxD, RxD, WR, RD) |

## Special Functions of Port 3 Pins:

| Pin | Alternate Function |
|---|---|
| P3.0 | RxD (Serial Input) |
| P3.1 | TxD (Serial Output) |
| P3.2 | INT0 (External Interrupt 0) |
| P3.3 | INT1 (External Interrupt 1) |
| P3.4 | T0 (Timer 0 input) |
| P3.5 | T1 (Timer 1 input) |
| P3.6 | $W\overline{R}$ (External Write) |
| P3.7 | $R\overline{D}$ (External Read) |

## 5. With Diagrams Explain the Internal RAM Structure of 8051 Microcontroller

The 8051 microcontroller contains **128 bytes of internal RAM** (from **00H to 7FH**) and **128 bytes of Special Function Register (SFR)** memory (**80H to FFH**).

## Internal RAM (00H–7FH) Breakdown:

| Address Range | Memory Section | Description |
|---|---|---|
| 00H–1FH | Register banks (Bank 0–3) | R0–R7 in 4 banks (8 registers × 4 banks = 32 bytes) |
| 20H–2FH | Bit-addressable RAM | 128 individual bits can be set/cleared |
| 30H–7FH | General-purpose RAM | Read/Write memory for variables, stack, etc. |

## Diagram: Internal RAM Organization (128 Bytes)

```
00H ──────► Bank 0 (R0-R7)
08H ──────► Bank 1
10H ──────► Bank 2
18H ──────► Bank 3
20H ──────► Bit-addressable RAM (20H to 2FH)
30H ──────► General-purpose RAM (up to 7FH)
```

- **Bit-addressable** region (20H–2FH) allows direct bit operations like `SETB`, `CLR`.
- **General RAM** is used for stack (default from 08H upward).

Register bank selection controlled by **RS1, RS0** bits in PSW.

6. Bit Pattern of PSW (Program Status Word) and Explanation of Flag Bits

## PSW Register (D0H):

- An **8-bit register** holding status flags and control bits.
- Affects instruction execution and conditional branching.

## Bit Pattern of PSW:

| **Bit No** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
|---|---|---|---|---|---|---|---|---|
| Name | CY | AC | F0 | RS1 | RS0 | OV | — | P |

- **CY** – Carry Flag
- **AC** – Auxiliary Carry (used in BCD operations)
- **F0** – User-defined flag
- **RS1, RS0** – Register bank select bits
- **OV** – Overflow Flag
- **—** – Reserved
- **P** – Parity Flag

## Flag Explanation:

| Flag | Name | Function |
|---|---|---|
| CY | Carry | Set when carry occurs in MSB bit |
| AC | Auxiliary Carry | Set when carry occurs from bit 3 to 4 |
| OV | Overflow | Set if arithmetic overflow in signed ops |
| P | Parity | Set if result has even number of 1s |
| RS1, RS0 | Register Bank Select | 00: Bank 0, 01: Bank 1, etc. |
| F0 | General-purpose Flag | Can be used in logic ops |

## Example: If `PSW = 11000000B`:

- CY = 1 (carry occurred)
- AC = 1 (aux carry occurred)
- Others = 0

## 7. With Circuit Diagram Explain Function of Port 0 and Port 1

## Port 0 (P0.0 to P0.7)

| Function | Dual-purpose |
|---|---|
| General I/O | Yes |
| Address/Data Lines (AD0–AD7) | When external memory is used |

- **Requires external pull-up resistors** when used as I/O.
- Used for **multiplexed address/data bus** in external memory interface.

### Port 1 (P1.0 to P1.7)

| Function | Dual-purpose |
|---|---|
| General I/O | No (dedicated) |

- True bidirectional port.
- **No external pull-up required**.
- Directly usable as input/output.

### Circuit Diagram Description:

**Each port pin** consists of:

- **Output latch**
- **Input buffer**
- **FET driver** and **pull-up resistors**

**Port 0 Diagram:**

```
P0.x → Open-drain → requires external pull-up resistor
       ↘ connected to ALE / ADx when memory is used
```

**Port 1 Diagram:**

```
P1.x → Internal pull-up + latch → standard TTL logic
       ↘ directly usable as digital input/output
```

### Key Differences Between Port 0 and Port 1:

| Feature | Port 0 | Port 1 |
|---|---|---|
| Internal Pull-up | No | Yes |
| External Memory Interface | Used (AD0–AD7) | Not used |
| Default Use | I/O & Multiplexed bus | I/O only |

**8. Criteria for Choosing a Microcontroller for an Application**

When selecting a microcontroller for a specific embedded application, consider the following **technical and economic factors**:

### Processing Power / Clock Speed

- Depends on computational complexity.
- For real-time or signal processing: higher MHz/GHz needed.

### Memory (RAM/ROM/Flash)

- Sufficient program memory (Flash/ROM) and data memory (RAM).
- Applications with data logging or large tables need more.

### I/O Ports

- Number and type of **digital, analog, PWM, UART, SPI, I2C** ports.

### Power Consumption

- Important for **battery-powered devices**.
- Choose low-power variants (e.g., sleep modes).

### Peripheral Support

- On-chip timers, ADC, DAC, comparators, USB, CAN, etc.
- Reduces need for external components.

### Cost

- Must match **budget** constraints of the application.

### Development Tools & Ecosystem

- Availability of **IDE**, compilers, programmers/debuggers.
- Community support and documentation.

### Packaging & Size

- DIP/SOP/QFP packages based on board space and manufacturing needs.

### Reliability & Environmental Tolerance

- For industrial or automotive, support for wide temperature range and EMI resistance.

### Example Use Case:

For a **temperature data logger**:

- Low-power, ADC input, small flash, UART for display
- Choose: **ATmega328P (used in Arduino Uno)**

9. Explain different addressing modes available in 8051 with examples and name the addressing modes of the following instructions i) MOV F0H, #29H ii) ADD A, 30H iii) MOV 35H, @R0 iv) SUBB A, R1 v) XRL A, @R1.

Addressing modes determine **how an operand is accessed** in an instruction.

### Types of Addressing Modes in 8051:

| Mode | Description | Example |
|---|---|---|
| **Immediate** | Data is provided in instruction | `MOV A, #45H` |
| **Register** | Operand is a register | `ADD A, R0` |
| **Direct** | Directly refers to internal RAM or SFR | `MOV A, 30H` |

| Mode | Description | Example |
|---|---|---|
| **Register Indirect** | Address stored in a register (R0 or R1) | `MOV A, @R0` |
| **Indexed** | Used with DPTR or PC + accumulator | `MOVC A, @A+DPTR` |

## Identify the Addressing Modes:

| Instruction | Addressing Mode | Explanation |
|---|---|---|
| i) `MOV F0H, #29H` | Immediate | Moves constant 29H to location F0H |
| ii) `ADD A, 30H` | Direct | Adds content at memory address 30H to A |
| iii) `MOV 35H, @R0` | Register Indirect | Moves data from address pointed by R0 to 35H |
| iv) `SUBB A, R1` | Register | Subtracts R1 + borrow from A |
| v) `XRL A, @R1` | Register Indirect | Performs XOR between A and value at [R1] |

## 10. **What is Stack and Stack Pointer? Explain PUSH and POP with Example**

### Stack in 8051

- A special area in **internal RAM** (usually starts at 08H).
- Works in **LIFO (Last In First Out)** order.
- Stores temporary data like return addresses, register values.

### Stack Pointer (SP)

- An **8-bit register** that holds address of **top of stack**.
- **Default value = 07H** after reset.
- **Auto-increments on PUSH** and **auto-decrements on POP**.

### PUSH Instruction

- Syntax: `PUSH source`
- Stores the content of a register or memory location onto the stack.
- SP = SP + 1 → value stored at [SP]

**Example:**

```
MOV A, #25H
PUSH ACC     ; SP increments, A pushed onto stack
```

### POP Instruction

- Syntax: `POP destination`
- Retrieves data from the stack into a register.
- Destination = [SP]; SP = SP – 1

**Example:**

```
POP B        ; Value at top of stack is moved to B
```

## Stack Operation Flow:

| Step | Instruction | SP Value | Stack Content |
|---|---|---|---|
| 1 | Reset | 07H | — |
| 2 | PUSH ACC | 08H | Stack[08] = A |
| 3 | PUSH B | 09H | Stack[09] = B |
| 4 | POP C | 08H | C = Stack[09] |

11. Define assembler directives. With example explain all the assembler directives supported by 8051 microcontrollers.

## Definition:

Assembler directives are instructions **for the assembler**, not for the microcontroller. They help in organizing code, declaring data, and defining memory layout.

## Common Assembler Directives:

| Directive | Description | Example |
|---|---|---|
| **ORG** | Sets start address | `ORG 0000H` |
| **DB** | Define Byte (8-bit constant or ASCII) | `DATA DB 55H, MSG DB 'HELLO'` |
| **DW** | Define Word (16-bit constant) | `COUNT DW 0A95H` |
| **EQU** | Assigns constant value to symbol | `PI EQU 3.14` |
| **END** | End of program (mandatory) | `END` |

## Additional:

- **CODE**: Defines code segment
- **DATA**: Declares data area

**Assembler Directives help to:**

- Allocate memory
- Organize structure
- Improve readability and maintainability

## 12. Explain the Following Instructions with Example

## i) SWAP A

- Swaps **nibbles** in the accumulator.
- High nibble ↔ Low nibble

**Example:**

```
MOV A, #3CH  ; A = 0011 1100
SWAP A       ; A = 1100 0011 = C3H
```

### ii) RRC A

- **Rotate right** through **carry flag**
- Bit 0 → Carry, Carry → Bit 7

**Example:**

```
MOV A, #95H  ; A = 1001 0101
RRC A        ; A = 1100 1010, CY = 1
```

### iii) DIV AB

- Divides A by B → Quotient in A, Remainder in B
- `A = A / B`, `B = A mod B`

**Example:**

```
MOV A, #0AH
MOV B, #03H
DIV AB       ; A = 03H, B = 01H
```

### iv) XCHD A, @Ri

- Exchanges **lower nibble** of A with that of memory pointed by R0/R1

**Example:**

```
MOV A, #5CH     ; A = 0101 1100
MOV R0, #40H
MOV 40H, #3AH   ; Memory = 0011 1010
XCHD A, @R0     ; A = 0101 1010, 40H = 0011 1100
```

### v) DA A (Decimal Adjust Accumulator)

- Adjusts result in A after **BCD addition**
- Used after `ADD` when operands are BCD

**Example:**

```
MOV A, #29H
ADD A, #25H    ; A = 4EH → Not valid BCD
DA A           ; A adjusted to 54H (valid BCD)
```

## 13. Check the Correctness of Instructions – Correct If Wrong

### 1. `CJNE @RI, #D_ADDRESS, REL8`

### ✕ Incorrect

- **@Ri** (R0 or R1) is allowed only with **byte** data (not direct address).
- `#D_ADDRESS` implies a direct address which cannot be compared directly with @Ri.

**Correct:**

```
CJNE @R0, #DATA, REL8
```

**2.** **`ADDC @RI, A`**

### ✕ Incorrect Order

- Accumulator **must be first operand** in arithmetic operations in 8051.

**Correct:**

```
ADDC A, @R0
```

**3.** **`DJNZ #DATA, REL8`**

### ✕ Incorrect

- **#DATA** is immediate data. DJNZ cannot operate on immediate values.
- Only **registers** or **direct addresses** are allowed.

**Correct:**

```
DJNZ R2, REL8
```

or

```
DJNZ 30H, REL8
```

**4.** **`MOVX @DPTR, R1`**

### ✕ Incorrect

- `MOVX @DPTR` can only be used with **A (Accumulator)**.

**Correct:**

```
MOVX @DPTR, A
```

**5.** **`MOV R0, R1`**

**Correct**

- Legal instruction. Moves contents of R1 to R0.

**6.** **`MOV A, DPTR`**

### ✕ Incorrect

- You **cannot move DPTR directly to A**. Need to use indirect move.

**Correct:**

```
MOV A, DPH
; Or use:
MOV A, #LOW(DPTR) ; using assembler macros if supported
```

**7.** **`MOV A, @R3`**

✕ **Incorrect**

- Only **R0 or R1** can be used with indirect addressing (`@Ri`)

**Correct:**

```
MOV A, @R0
```

## 14. Assembly Program to Toggle Bits of Port P1 with Delay

```
ORG 0000H

START:
    MOV P1, #0FFH      ; Set P1 high
    ACALL DELAY
    MOV P1, #00H       ; Set P1 low
    ACALL DELAY
    SJMP START         ; Repeat forever

DELAY:
    MOV R2, #255
LOOP1:
    MOV R1, #255
LOOP2:
    DJNZ R1, LOOP2
    DJNZ R2, LOOP1
    RET

END
```

**Explanation**:

- Port P1 toggled high and low with a software delay loop.
- `ACALL` used for subroutine.
- `SJMP` used for infinite loop.

## 15. Difference Between JMP and CALL Instructions with Program Range

### JMP (Jump)

- Transfers control to another **part of program** without return.
- No stack involvement.
- Used for **loops and branching**.

**Example:** `JMP LABEL`

### CALL

- Transfers control to **subroutine** and saves return address on the **stack**.
- Must be followed by `RET`.

**Example:**

```
CALL DELAY
...
RET
```

### Comparison Table

| Feature | JMP | CALL |
|---|---|---|
| Return Address | Not stored | Stored on stack |
| RET used | No | Yes |
| Use Case | Branching, looping | Subroutine execution |
| Stack Use | No | Yes |

## Program Range

| Instruction | Range |
|---|---|
| **SJMP** | ±128 bytes |
| **LJMP** | 64K full address |
| **AJMP** | Within same 2KB page |
| **ACALL** | Within same 2KB page |
| **LCALL** | 64K full address |

## Diagram (Conceptual):

```
Main Program
 ↓
CALL → Subroutine → RET → Return
 ↓
JMP → Label (direct jump, no return)
```

## 16. Assembly Program to Copy 55H to 5 Memory Locations (200H–204H)

### a) Using Direct Addressing Mode

```
MOV 0200H, #55H
MOV 0201H, #55H
MOV 0202H, #55H
MOV 0203H, #55H
MOV 0204H, #55H
```

### b) Using Register Indirect Without Loop

```
MOV DPTR, #0200H
MOV A, #55H
MOVX @DPTR, A
INC DPTR
MOVX @DPTR, A
INC DPTR
MOVX @DPTR, A
INC DPTR
MOVX @DPTR, A
INC DPTR
MOVX @DPTR, A
```

### c) Using Register Indirect With Loop

```
MOV DPTR, #0200H
MOV R0, #05       ; Counter
MOV A, #55H
```

```
COPY:
    MOVX @DPTR, A
    INC DPTR
    DJNZ R0, COPY
```

**Note**:

- `MOVX` is used to access **external memory**.
- Replace `MOVX` with `MOV` if using **internal RAM** like 30H.

## 17. Assembly Program to Read from P1 and:

- If **odd**, find **complement** and send to **P2**
- If **even**, send as-is to **P3 or**

**Write an assembly level program to read the number from P1 if it is odd find the complement of the number and send it through P2 otherwise send it through P3.**

## 8051 Assembly Program:

```
ORG 0000H

START:
    MOV A, P1           ; Read input from P1
    ANL A, #01H         ; Mask LSB to check if number is odd
    JZ EVEN             ; If zero → even number

    ; --- Odd case ---
    MOV A, P1           ; Read again
    CPL A               ; Complement
    MOV P2, A           ; Send to P2
    SJMP END

EVEN:
    MOV A, P1
    MOV P3, A           ; Send even number to P3

END: SJMP END           ; Infinite loop

END
```

## Explanation:

- `ANL A, #01H` checks LSB.
- `CPL A` complements A (bitwise NOT).
- Branches to P2 if odd, else to P3.

## 18: Find the Time Required to Execute the Following Instructions

## Formula:

Each machine cycle = 12Crystal Frequency (in Hz)

## Instruction 1: `ADD A, #54H`

- Machine cycles = 1
- Crystal Frequency = 11.0592 MHz
- Time = 1211.0592×106=1.085 µs\frac{12}{11.0592 \times 10^6} = 1.085 \, \mu s

### Instruction 2: `XRL 35, #47H`

- Machine cycles = 2
- Crystal = 12 MHz
- Time = 2×1212×106=2 µs2 \times \frac{12}{12 \times 10^6} = 2 \, \mu s

### Instruction 3: `MUL AB`

- Machine cycles = 3
- Crystal = 11.0592 MHz
- Time = 3×1211.0592×106=3.255 µs3 \times \frac{12}{11.0592 \times 10^6} = 3.255 \, \mu s

### Instruction 4: `MOVC A, @A+DPTR`

- Machine cycles = 4
- Crystal = 12 MHz
- Time = 4×1212×106=4 µs4 \times \frac{12}{12 \times 10^6} = 4 \, \mu s

### Summary Table

| Instruction | Machine Cycles | Crystal Frequency | Time Required |
|---|---|---|---|
| ADD A, #54H | 1 | 11.0592 MHz | 1.085 µs |
| XRL 35, #47H | 2 | 12 MHz | 2.000 µs |
| MUL AB | 3 | 11.0592 MHz | 3.255 µs |
| MOVC A, @A+DPTR | 4 | 12 MHz | 4.000 µs |

## 19: Analyze the Given Assembly Program

### Code Breakdown:

```
ORG 0011H         ; Set code starting address at 0011H
MOV R2, #0AH      ; R2 = 10 (loop counter)
MOV R0, #20H      ; R0 = 20H (source address)
MOV R1, #30H      ; R1 = 30H (destination address)

Here:
  MOV A, @R0      ; A = data from [R0] (20H, 21H...)
  MOV @R1, A      ; Store A to [R1] (30H, 31H...)
  INC R0          ; Next source address
  INC R1          ; Next destination address
  DJNZ R2, Here   ; Repeat 10 times
END
```

### What the Program Does:

- **Copies 10 bytes** from memory starting at address **20H** to memory starting at **30H**.
- Uses **R0 as source pointer**, **R1 as destination pointer**, and **R2 as counter**.
- Efficient data block transfer loop.

### Final Effect:

- After execution:
  `RAM[30H to 39H] = RAM[20H to 29H]`
- **Registers after execution**:
  - R0 = 2AH, R1 = 3AH, R2 = 00H

## 20: Calculate Time Delay of the Given Loop

### Subroutine:

```
DELAY:    MOV  R3, #255   ; 1 machine cycle
HERE:     NOP             ; 1
          NOP             ; 1
          NOP             ; 1
          NOP             ; 1
          DJNZ R3, HERE   ; 2
```

### Total Machine Cycles per Loop Iteration:

- 4 × NOP = 4
- DJNZ = 2
- Total per iteration = **6 machine cycles**

### Total Iterations:

R3 = 255 → loop runs **255 times**

### Total Machine Cycles = 255 × 6 = 1530 cycles

### Clock Period:

Machine cycle time=1211.0592×106≈1.085µs

### Time Delay:

Total delay=1530×1.085µs≈1.659 ms

### Final Answer:

**The time delay is approximately 1.659 milliseconds**

## 21. Count Number of 1's and 0's in 8-bit Data from P1

- Store **count of 1s** in **30H**
- Store **count of 0s** in **31H**

### Assembly Code:

```
ORG 0000H

START:
    MOV A, P1          ; Read 8-bit data
    MOV R0, #08        ; Counter for 8 bits
    MOV R1, #00        ; Count of 1s
    MOV R2, A          ; Store original value in R2

COUNT_LOOP:
    RRC A              ; Rotate right through carry → LSB into CY
    JNC NEXT           ; If CY=0, skip increment

    INC R1             ; If CY=1, increment count of 1s

NEXT:
    DJNZ R0, COUNT_LOOP

    MOV 30H, R1        ; Store count of 1s
```

```
    MOV A, #08
    SUBB A, R1          ; 8 - number of 1s = number of 0s
    MOV 31H, A

HERE: SJMP HERE

END
```

### Explanation:

- `RRC` shifts right and puts LSB into Carry.
- `JNC` checks if Carry is clear (0).
- Finally, `SUBB` gives count of 0s.

## Module 4 – 1. Difference Between Timer and Counter + TMOD Register

### Timer vs Counter (8051)

| Feature | Timer | Counter |
|---|---|---|
| Function | Measures **time intervals** | Counts **external events/pulses** |
| Clock Source | Internal clock | External pulses (T0/T1 pins) |
| Use Case | Delay generation | Event counting |
| Control Bit | `C/T` = 0 | `C/T` = 1 |

### TMOD Register (Timer Mode Register)

- **8-bit register** divided into two 4-bit fields:
  - Lower 4 bits for **Timer 0**
  - Upper 4 bits for **Timer 1**

### Format:

```
TMOD: | GATE | C/T | M1 | M0 | GATE | C/T | M1 | M0 |
          ↑      ↑     ↑    ↑     ↑      ↑     ↑    ↑
      Timer1                    Timer0
```

| Bit | Meaning |
|---|---|
| M0, M1 | Select Mode (00=13-bit, 01=16-bit, 10=8-bit auto-reload) |
| C/T | 0 = Timer, 1 = Counter |
| GATE | 0 = Always on, 1 = Enabled by INTx pin |

## 2. TCON, SCON, and PCON Registers Explained

### TCON (Timer Control Register)

| Bit | Symbol | Function |
|---|---|---|
| 7 | TF1 | Timer 1 overflow flag |
| 6 | TR1 | Timer 1 run control |
| 5 | TF0 | Timer 0 overflow flag |

| Bit | Symbol | Function |
|---|---|---|
| 4 | TR0 | Timer 0 run control |
| 3 | IE1 | External Interrupt 1 flag |
| 2 | IT1 | INT1 type: 1=edge, 0=level |
| 1 | IE0 | External Interrupt 0 flag |
| 0 | IT0 | INT0 type: 1=edge, 0=level |

## SCON (Serial Control Register)

| Bit | Symbol | Function |
|---|---|---|
| 7 | SM0 | Serial mode select bit 0 |
| 6 | SM1 | Serial mode select bit 1 |
| 5 | SM2 | Multiprocessor communication |
| 4 | REN | Receive Enable |
| 3 | TB8 | Transmit bit 8 (mode 2/3) |
| 2 | RB8 | Receive bit 8 (mode 2/3) |
| 1 | TI | Transmit Interrupt flag |
| 0 | RI | Receive Interrupt flag |

## PCON (Power Control Register)

| Bit | Symbol | Function |
|---|---|---|
| 7 | SMOD | Double baud rate if set |
| 1 | PD | Power-down mode (1 = on) |
| 0 | IDL | Idle mode (1 = on) |

- **SMOD** = 1 doubles baud rate in serial mode 1, 2, or 3.

3. Illustrate the steps to be followed for using Timer 1 in mode 2 and also find the value to be stored in Register TH1 to get a delay of 100 microseconds, if the crystal frequency is 11.0592 MHz.

## Steps to Use Timer 1 in Mode 2

1. **Select Mode 2** using **TMOD register**:
   - Timer 1 = upper 4 bits of TMOD
   - Set `TMOD = 0x20` → 0010 0000 (Mode 2 for Timer 1)
2. **Load TH1 with reload value** (auto-loads into TL1 after overflow)
3. **Start Timer** using `TR1` in TCON register:
   - `SETB TR1` or `TCON = TCON | 0x40`
4. **Wait for overflow flag (optional)**:

- `TF1` in TCON indicates overflow.

5. **Stop Timer** (if needed):
   - `CLR TR1`

### Calculate TH1 for 100 µs Delay

**Given**:

- Crystal frequency = **11.0592 MHz**
- One machine cycle = 12 clock cycles
  → Time per machine cycle = 1.085 µs
- Mode 2: **8-bit auto-reload**
- Timer ticks every 1.085 µs

### Number of counts for 100 µs:

1001.085≈92 counts\frac{100}{1.085} \approx 92 \text{ counts}

TH1 counts from initial value to **255** (FFH), so:

TH1=256−92=164⇒A4H\text{TH1} = 256 - 92 = 164 \Rightarrow A4H

### Final Timer Setup:

```
MOV TMOD, #20H   ; Timer 1, Mode 2
MOV TH1, #0A4H   ; Load reload value
MOV TL1, #0A4H   ; Initial value
SETB TR1         ; Start Timer
```

## 4. Serial vs Parallel Communication + Asynchronous Serial Communication with an Example

### Comparison Table

| Feature | Serial Communication | Parallel Communication |
|---|---|---|
| Data Transmission | 1 bit at a time | Multiple bits simultaneously |
| Speed | Slower for large data | Faster for short-distance data |
| Cost | Lower (fewer wires) | Higher (more lines needed) |
| Distance | Suitable for long distances | Limited to short distances |
| Examples | RS-232, UART, USB | Data buses in microcontrollers |

### Asynchronous Serial Communication

- No clock line shared.
- Uses **start and stop bits** to define data boundaries.
- Optional **parity bit** for error checking.
- Data is framed as:

```
START BIT → DATA BITS (LSB first) → PARITY (optional) → STOP BIT
```

### Example: Transmit ASCII 'A'

- ASCII 'A' = 0x41 = `01000001`
- Frame = `0` (Start) + `01000001` + `1` (Stop)

Transmitted bitstream: `0 1 0 0 0 0 0 1 1`

## 5. C Program to Send "IOT" Serially at 9600 Baud (8051)

Or

**Write and explain a C Program for 8051 to transfer the message "IOT" serially at 9600 Baud continuously. Use 8-bit data and 1 stop bit.**

### Assumptions:

- 8-bit data, 1 stop bit
- 11.0592 MHz crystal
- Mode 1 (8-bit UART)
- Baud rate = 9600

### Formula for TH1:

TH1=256−(Timer_Freq32×Baud_Rate)

Timer freq = 11.0592 MHz / 12 = 921.6 kHz

TH1=256−(92160032×9600)=256−3=253=FDH

Program:

```
#include <reg51.h>

void main() {
    SCON = 0x50;       // Mode 1: 8-bit UART, REN enabled
    TMOD = 0x20;       // Timer 1, Mode 2
    TH1 = 0xFD;        // Load baud rate value for 9600
    TR1 = 1;           // Start timer

    while(1) {
        SBUF = 'I';
        while (TI == 0);  // Wait for transmit
        TI = 0;

        SBUF = 'O';
        while (TI == 0);
        TI = 0;

        SBUF = 'T';
        while (TI == 0);
        TI = 0;
    }
}
```

### Explanation:

- `SCON = 0x50`: enables UART, 8-bit data, and receive enable.
- Timer 1 generates baud rate via auto-reload.
- Message "IOT" sent repeatedly.

6. RS-232 Handshaking Signals and Role of MAX232

Or

Justify and explain the need of RS-232 hand shaking signals and specify the purpose of MAX – 232 while interfacing 8051 with the RS 232.

## RS-232 Interface:

Standard for **serial communication between DTE (Data Terminal Equipment)** and **DCE (Data Communication Equipment)**.

## Handshaking Signals:

| Signal | Name | Function |
|---|---|---|
| **RTS** | Request to Send | Host wants to send data |
| **CTS** | Clear to Send | Receiver ready to accept data |
| **DTR** | Data Terminal Ready | Terminal ready for communication |
| **DSR** | Data Set Ready | Modem or device is ready |
| **DCD** | Data Carrier Detect | Connection established |

## Why Use MAX232?

| Feature | Description |
|---|---|
| Voltage Level | Converts **TTL (0–5V) ↔ RS-232 levels (±12V)** |
| Usage | Needed because 8051 uses TTL, but RS-232 uses higher voltage swings |
| Capacitors | Requires external capacitors to generate voltage levels |
| Role | Acts as a **bridge** between microcontroller and PC/modem |

## Interfacing Diagram:

```
8051 Tx (P3.1) → T1IN → MAX232 → T1OUT → PC Rx
8051 Rx (P3.0) ← R1OUT ← MAX232 ← R1IN ← PC Tx
```

## Summary:

- **RS-232 handshaking ensures reliable serial transfer**
- **MAX232 enables voltage compatibility between 8051 and RS-232 devices**

7. Generate a 10 kHz Square Wave on P1.4 Using Timer 0 Mode 2 (C and Assembly)

## Requirements:

- Frequency = 10 kHz → Time period = 100 µs
- Square wave → High and Low each = 50 µs
- Clock = 22 MHz → 1 machine cycle = 12 / 22MHz = 0.545 µs
- Delay needed = 50 µs → Counts = 50 / 0.545 ≈ **92**

## C Program:

```
#include <reg51.h>

void timer_delay() {
    TH0 = 256 - 92;   // Load value = 256 - 92 = 164 = A4H
    TL0 = 256 - 92;
    TR0 = 1;          // Start timer
    while (TF0 == 0); // Wait for overflow
    TR0 = 0;
    TF0 = 0;
}

void main() {
    TMOD = 0x02;      // Timer 0, Mode 2 (8-bit auto reload)
    TH0 = 256 - 92;

    while (1) {
        P1 ^= 0x10;   // Toggle P1.4
        timer_delay();
    }
}
```

## Assembly Program:

```
ORG 0000H
START:
    MOV TMOD, #02H     ; Timer 0, mode 2
    MOV TH0, #0A4H     ; 256 - 92
    MOV TL0, #0A4H
MAIN_LOOP:
    CPL P1.4
    SETB TR0
WAIT:
    JNB TF0, WAIT
    CLR TR0
    CLR TF0
    SJMP MAIN_LOOP
END
```

8. Counter 1 in Mode 2 to Count External Pulses from T1 (P3.5)

Or

Assume that a 1-Hz external clock is being fed into pin T1(P3.5). Write a C programming for counter 1 in mode-2(8-bit auto reload) to count up and display the state of the TL1 count on P1. Start the count at 00H.

## Setup:

- Use **Timer 1 in counter mode**
- Mode 2 → 8-bit auto-reload
- Clock source = External pulses on **T1 (P3.5)**

## C Program:

```
#include <reg51.h>

void main() {
    TMOD = 0x60;        // Timer1, Mode 2, Counter mode (C/T = 1)
    TH1 = 0x00;         // Reload value = 0
    TL1 = 0x00;
    TR1 = 1;            // Start counter

    while(1) {
        P1 = TL1;       // Display count value on Port 1
    }
}
```

9. Generate Square Wave on P0: tON = 3ms, tOFF = 7ms Using Timer 0 Mode 1

Or

Write 8051 program to generate square wave with tON = 3ms and tOFF = 7ms on all pins of Port 0. System Clock is 22MHz. Use timer0 in Mode-1.

## Calculation:

- tON = 3 ms, tOFF = 7 ms
- 1 machine cycle = 12 / 22 MHz = 0.545 µs

**Delay counts:**

3 ms=3000/0.545≈5500⇒65536−5500=60036=EA74H7 ms=7000/0.545≈12844⇒65536−12844=52692=CD74H

## C Program:

```
#include <reg51.h>

void delay(unsigned int high, unsigned int low) {
    TH0 = high;
    TL0 = low;
    TF0 = 0;
    TR0 = 1;
    while (TF0 == 0);
    TR0 = 0;
}

void main() {
    TMOD = 0x01;  // Timer 0, Mode 1

    while (1) {
        P0 = 0xFF;          // ON
        delay(0xEA, 0x74);  // 3 ms

        P0 = 0x00;          // OFF
        delay(0xCD, 0x74);  // 7 ms
    }
}
```

10. Steps for Programming 8051 for Serial Data Transfer and Reception

Step-by-Step Procedure:

1. **Select Serial Mode (SCON Register)**
   - Common: Mode 1 → 8-bit UART with start and stop bits
   - `SCON = 0x50` → SM0=0, SM1=1, REN=1, others = 0
2. **Set Baud Rate Using Timer 1**
   - Use **Timer 1 in Mode 2 (Auto-reload)**
   - Calculate TH1 using:

     TH1=256−(Crystal_Freq384×Baud_Rate)

     For 9600 baud @ 11.0592 MHz → `TH1 = 0xFD`

3. **Start Timer 1**
   - `TMOD = 0x20`, `TR1 = 1`
4. **Enable Serial Interrupts (optional)**
   - `IE |= 0x90` (enable serial + global interrupt)

### Transmit a Character:

```
SBUF = 'A';          // Load data
while (TI == 0);     // Wait for transmission to complete
TI = 0;              // Clear transmit interrupt flag
```

### Receive a Character:

```
while (RI == 0);     // Wait for data
char data = SBUF;    // Read received byte
RI = 0;              // Clear receive interrupt flag
```

### Summary:

| Step | Register / Bit Used |
|---|---|
| Serial Mode | SCON (SM0, SM1) |
| Baud Rate | TH1 (via Timer 1) |
| Enable Tx/Rx | REN in SCON |
| Start Timer | TR1 in TCON |
| Send/Receive | SBUF register |
| Flags | TI, RI |

Module5 1. Compare Polling and Interrupt. What Happens When a Microcontroller is Interrupted?
Or

Compare Polling and Interrupt. What are the steps a microcontroller performs upon a activation of Interrupt.

### Comparison Between Polling and Interrupt

| Feature | Polling | Interrupt |
|---|---|---|
| Method | CPU continuously checks status flags | Device alerts CPU via interrupt signal |
| Efficiency | Less efficient; CPU wastes cycles | Efficient; CPU does useful work until needed |
| Response Time | Slower (depends on polling rate) | Faster (immediate response) |
| Power Consumption | Higher (CPU always active) | Lower (CPU can sleep until interrupt) |
| Complexity | Simpler to implement | Requires ISR and interrupt configuration |
| Example | Checking switch input in a loop | UART receive complete interrupt |

### Steps on Interrupt Activation in 8051:

1. **Current instruction finishes.**
2. **Interrupt type is identified** (INT0, T0, INT1, T1, Serial).
3. **PC (Program Counter)** is pushed to stack.
4. **Program jumps to ISR** (interrupt service routine) address from vector table.
5. **ISR executes**; optionally disables further interrupts (if not nested).

6. **Return from ISR** using `RETI`:
   - Pops PC from stack.
   - Resumes main program.

## 2. Instructions to Enable/Disable Interrupts

### a) Enable Serial Interrupt, Timer 0 Interrupt, External Interrupt 1

```
MOV IE, #10010110B  ; Enable EA (bit 7), EX1 (bit 2), ET0 (bit 1), ES (bit 4)
```

| Bit | Function |
|---|---|
| 7 | EA (Enable All) |
| 4 | ES (Serial) |
| 2 | EX1 |
| 1 | ET0 |

### b) Disable Timer 0 Interrupt (Clear ET0 only)

```
CLR ET0    ; Bit 1 of IE
```

### c) Disable All Interrupts in One Instruction

```
CLR EA     ; EA = 0 disables all interrupt sources
```

## 3. Importance of TI and RI Flags

### TI (Transmit Interrupt)

- Set when a byte is **transmitted** via UART (SBUF).
- Must be **cleared in software** after checking.
- **Indicates** transmitter is ready to send next byte.

**Usage:**

```
while (TI == 0);  // Wait until transmission complete
TI = 0;
```

### RI (Receive Interrupt)

- Set when a byte is **received** via UART.
- Must be cleared in software.
- **Indicates** data is ready to be read from SBUF.

**Usage:**

```
while (RI == 0);  // Wait until data received
data = SBUF;
RI = 0;
```

### Summary Table:

| Flag | Trigger Event | Set by | Cleared by | Use |
|---|---|---|---|---|
| TI | Transmit complete | UART | Software | Load next byte |
| RI | Receive complete | UART | Software | Read incoming byte |

## 4. Interfacing DAC with 8051 (with Program & Diagram)

### Concept:

- DAC converts digital value from microcontroller to analog voltage.
- 8-bit DAC like **DAC0808** or **MC1408** can be used.
- DAC requires:
  - 8-bit data input (e.g., from Port 1)
  - Control signal ($\overline{WR}$ or STB depending on DAC type)

### Basic Circuit Diagram Description:

```
8051 P1.0–P1.7  →  D0–D7 of DAC0808
Control Line     →  WR‾ or STB of DAC
DAC Output       →  Low-pass filter → Analog Output
```

- **Vref+** connected to +5V, **Vref–** to GND
- **Analog output** from DAC is connected to oscilloscope or speaker

### Assembly Program to Output Ramp Wave to DAC

```
ORG 0000H
START:
    MOV A, #00H
LOOP:
    MOV P1, A         ; Send data to DAC via Port 1
    ACALL DELAY
    INC A
    CJNE A, #0FFH, LOOP
    SJMP START

DELAY:
    MOV R2, #255
D1: MOV R1, #255
D2: DJNZ R1, D2
    DJNZ R2, D1
    RET
END
```

### Explanation:

- Ramp signal from 0 to 255 sent to DAC.
- Delay loop controls ramp rate (frequency).
- Could generate triangular or sine-like wave using lookup tables.

## 5. Stepper Motor Interfacing with 8051 – Diagram, Calculation, and C Code

Or

Explain about stepper motor interface with diagram, and also write a c program if motor takes 90 steps to complete one revolution and show the calculation (Clockwise Direction).

## Stepper Motor Basics

- A **stepper motor** rotates in discrete steps.
- Each **step** turns the shaft by a fixed angle.
- Common types:
  - **Unipolar** (easy to control, common ground)
  - **Bipolar** (needs H-bridge driver)

## Diagram: Stepper Motor with ULN2003 and 8051

```
+5V
 |
[8051 Port P1] → IN1 to IN4 → ULN2003 Driver → Stepper Motor Coils A-B-C-D

Ex:
P1.0 → IN1
P1.1 → IN2
P1.2 → IN3
P1.3 → IN4
```

- **ULN2003**: Transistor driver used to interface 8051 with higher current motor coils.
- **Stepper Motor Coil Sequence (CW – 4-step)**:
- `1. 0001`
- `2. 0010`
- `3. 0100`
- `4. 1000`

## Stepper Motor Calculation

Assume:

- **90 steps per revolution**
- **1 step = 360° / 90 = 4°**
- To rotate **360°**, need 90 sequences.

## C Program for 360° Rotation (Clockwise)

```c
#include <reg51.h>

void delay() {
    int i, j;
    for (i = 0; i < 200; i++)
        for (j = 0; j < 1275; j++);  // ~1ms delay
}

void main() {
    unsigned char step_seq[4] = {0x01, 0x02, 0x04, 0x08};
    int i, step;

    for (step = 0; step < 90; step++) {
        for (i = 0; i < 4; i++) {
            P1 = step_seq[i];   // Send step pattern to motor
            delay();            // Wait for motor to step
        }
    }

    while (1);  // Stop here
}
```

6. Bit Contents of IE and IP Registers

## IE Register (Interrupt Enable) – Address: A8H

| Bit | Symbol | Function |
|---|---|---|
| 7 | EA | Enable all interrupts (1 = enable) |
| 6 | — | Reserved |
| 5 | ET2 | Timer 2 (8052 only) |
| 4 | ES | Serial port |
| 3 | ET1 | Timer 1 |
| 2 | EX1 | External interrupt 1 |
| 1 | ET0 | Timer 0 |
| 0 | EX0 | External interrupt 0 |

## IP Register (Interrupt Priority) – Address: B8H

| Bit | Symbol | Function |
|---|---|---|
| 7 | — | Reserved |
| 6 | — | Reserved |
| 5 | PT2 | Priority for Timer 2 |
| 4 | PS | Priority for Serial port |
| 3 | PT1 | Priority for Timer 1 |
| 2 | PX1 | Priority for External INT1 |
| 1 | PT0 | Priority for Timer 0 |
| 0 | PX0 | Priority for External INT0 |

If a bit is set (1), the interrupt gets **high priority**.

7. Changing Interrupt Priority Using IP Register

Or

Explain how interrupt priority can be changed using IP register. Also explain the default priorities assigned to interrupts in 8051 microcontrollers.

**Purpose:**

- IP register allows assigning **custom priorities** to interrupts.
- **1 = High priority**, **0 = Low priority**

### How to Change Priority:

Example: To give Timer 0 high priority:

```
MOV IP, #00000010B ; Set PT0 = 1
```

To give Serial interrupt and INT0 high priority:

```
MOV IP, #00010001B ; PS = 1, PX0 = 1
```

### Default Priorities in 8051 (when IP = 00H):

| Priority Level | Interrupt |
|---|---|
| 1 (Highest) | External INT0 |
| 2 | Timer 0 |
| 3 | External INT1 |
| 4 | Timer 1 |
| 5 (Lowest) | Serial |

To override, change respective bits in **IP register**.

## 8. Interrupts in 8051 and Their Vector Addresses

Or

Write the name of the Interrupt present in 8051 and indicate the starting address reserved in ROM for each interrupt.

The 8051 has **5 interrupt sources**, each with a **reserved ROM vector address**.

| Interrupt | Vector Address | Trigger Condition |
|---|---|---|
| External INT0 | 0003H | Falling edge at P3.2 (INT0) |
| Timer 0 | 000BH | Timer 0 overflow |
| External INT1 | 0013H | Falling edge at P3.3 (INT1) |
| Timer 1 | 001BH | Timer 1 overflow |
| Serial (Rx/Tx) | 0023H | TI or RI set |

### Summary Table:

| Interrupt | Mnemonic | Vector Address |
|---|---|---|
| External 0 | INT0 | 0003H |
| Timer 0 | T0 | 000BH |
| External 1 | INT1 | 0013H |
| Timer 1 | T1 | 001BH |
| Serial (UART) | Serial | 0023H |

Use `LJMP` at vector address to jump to actual ISR location:

```
ORG 0003H
LJMP EXT0_ISR
```

## 9. Keyboard Interface with 8051: Diagram, Scanning, and Key Identification

### Objective:

- Interface a **matrix keyboard** (typically 4×4 or 4×3) with 8051.
- Detect which key is pressed using **scanning technique**.

### Circuit Diagram (Description):

- Connect:
  - **Rows (R0–R3)** to **P1.0–P1.3**
  - **Columns (C0–C3)** to **P1.4–P1.7**
- Use pull-up resistors or configure unused pins as `1`.

```
    C0 C1 C2 C3
     |  |  |  |
R0 —●--●--●--●→ P1.0
R1 —●--●--●--●→ P1.1
R2 —●--●--●--●→ P1.2
R3 —●--●--●--●→ P1.3
     ↓  ↓  ↓  ↓
    P1.4–P1.7 (columns)
```

### Key Scanning Technique:

1. Set **rows = 0**, **columns = 1** (e.g., `P1 = 0xF0`).
2. Pull **one row LOW** at a time.
3. Read columns: if any bit becomes `0`, corresponding key is pressed.
4. Combine row and column to identify key using lookup table.

### Example Code Snippet (C):

```
#include <reg51.h>

void main() {
    unsigned char row, col;

    while (1) {
        for (row = 0; row < 4; row++) {
            P1 = ~(1 << row);        // Activate one row at a time
```

```
            col = P1 >> 4;           // Read column inputs
            if (col != 0x0F) {
                // Key press detected
                // Map row & column to specific key
            }
        }
    }
}
```

10. Monitor Switch (P2.7) to Control Stepper Motor Direction (C Program)

Or

A switch is connected to pin P2.7. write a C program to monitor the status of 'SW' and perform the following: a. If SW=0: the stepper motor moves clock wise If SW=1: the stepper motor moves counter clock wise.

**Assumptions:**

- Stepper motor connected to **Port 1**
- Clockwise and counter-clockwise use **different step sequences**

**C Program:**

```
#include <reg51.h>

void delay() {
    int i, j;
    for (i = 0; i < 200; i++)
        for (j = 0; j < 1275; j++);
}

void main() {
    unsigned char cw[]  = {0x01, 0x02, 0x04, 0x08};  // Clockwise
    unsigned char ccw[] = {0x08, 0x04, 0x02, 0x01};  // Counter-clockwise
    int i;

    while (1) {
        if (P2 & 0x80) {  // SW = 1 → P2.7 = 1
            for (i = 0; i < 4; i++) {
                P1 = ccw[i];
                delay();
            }
        } else {
            for (i = 0; i < 4; i++) {
                P1 = cw[i];
                delay();
            }
        }
    }
}
```

11. Interrupt Priority: MOV IP, #00001100B

Or

Assume that after reset, the interrupt priority is set by the instruction "MOV IP, #00001100B". Discuss the sequence in which the interrupts are serviced.

**IP Register After Instruction:**

```
Bit:  7 6 5 4 3 2 1 0
IP = 0 0 0 0 1 1 0 0  →  PT1 = 1, PX1 = 1
```

## Priority Assignment:

| Interrupt | Bit | Priority |
|---|---|---|
| Timer 1 (TF1) | PT1 | High |
| External INT1 (INT1) | PX1 | High |
| All others | | Low |

## Interrupt Service Sequence:

1. **If Timer 1 and INT1 occur together**:
   - Highest priority goes to the **lowest address vector** among same priority
   - INT1 (vector 13H) serviced before Timer 1 (1BH)
2. Remaining interrupts:
   - INT0, Timer 0, Serial → default priority, serviced in default order

12. Interfacing 16×2 LCD with 8051 (with Diagram & Code)

Or

With a neat diagram, explain the working of 16*2 LCD and write a program to interface the same with 8051 microcontrollers.

## LCD Basics:

- 16×2 LCD → 2 lines, 16 characters per line
- Pins: D0–D7 (data), RS (register select), RW, EN (enable)
- Connected in **8-bit mode or 4-bit mode**

## LCD Pin Connections (Example 8-bit mode):

| LCD Pin | Connected To |
|---|---|
| RS | P2.0 |
| RW | P2.1 |
| EN | P2.2 |
| D0–D7 | P1.0–P1.7 |

## Block Diagram:

```
8051 P1.0-P1.7 → LCD D0-D7
P2.0 → RS, P2.1 → RW, P2.2 → EN
```

## LCD Commands:

- `0x38` → 8-bit, 2-line
- `0x0C` → Display ON
- `0x06` → Auto-increment cursor
- `0x01` → Clear display

## Assembly Code Example (Partial):

```
ORG 0000H
MOV P2, #00H

; Initialize LCD
MOV A, #38H
ACALL LCD_CMD
MOV A, #0CH
ACALL LCD_CMD
MOV A, #06H
ACALL LCD_CMD

; Display 'H'
MOV A, #'H'
ACALL LCD_DATA

LCD_CMD:
    CLR P2.0     ; RS = 0
    CLR P2.1     ; RW = 0
    SETB P2.2    ; EN = 1
    MOV P1, A
    CLR P2.2     ; EN = 0
    ACALL DELAY
    RET

LCD_DATA:
    SETB P2.0    ; RS = 1
    CLR P2.1     ; RW = 0
    SETB P2.2    ; EN = 1
    MOV P1, A
    CLR P2.2
    ACALL DELAY
    RET

DELAY:
    MOV R1, #255
D1: MOV R2, #255
D2: DJNZ R2, D2
    DJNZ R1, D1
    RET

END
```

Let me know if you want:

- The same LCD interfacing program in **C**
- LCD interfacing using **4-bit mode**
- Stepper motor and LCD **Proteus simulation files**!

**8051 Microcontroller - Module 1 Detailed Notes**

## 1. Introduction to Microcontroller

**Definition:**
A microcontroller is an integrated circuit (IC) that combines a microprocessor core, memory (RAM, ROM), input/output ports, timers, and other peripherals on a single chip, designed to perform specific control tasks in embedded systems.

**Applications:**

- Home appliances
- Automotive electronics
- Industrial control systems
- Medical devices
- Consumer electronics

## 2. Microcontroller vs Microprocessor

| Feature | Microcontroller | Microprocessor |
|---|---|---|
| Components | CPU, RAM, ROM, I/O, Timers on-chip | Only CPU; needs external components |
| Application | Specific tasks | General-purpose computing |
| Power Consumption | Low | High |
| Speed | Moderate | High |
| Cost | Low | High |
| Size | Small | Larger (with peripherals) |

## 3. Harvard vs Von Neumann Architecture

**Harvard Architecture:**

- Separate memory and buses for program and data
- Simultaneous access possible
- Faster
- Example: 8051, PIC, AVR

**Von Neumann Architecture:**

- Shared memory and bus for program and data
- One-at-a-time access
- Slower
- Example: Traditional PC processors

## 4. RISC vs CISC Architecture

**RISC (Reduced Instruction Set Computer):**

- Simple instructions, each executes in 1 cycle
- Larger program size
- Easier to pipeline
- Example: ARM, AVR

**CISC (Complex Instruction Set Computer):**

- Complex, multi-cycle instructions
- Smaller programs
- Harder to pipeline
- Example: 8051, x86

## 5. 8051 Microcontroller Overview

**Developed by:** Intel in 1981
**Key Features:**

- 8-bit CPU
- Harvard architecture
- 4K ROM, 128B RAM
- 4 I/O ports (P0-P3)
- 2 Timers (Timer 0 & 1)
- 1 Serial port

- 5 Interrupt sources
- 64KB external code/data memory support

## 6. 8051 Architecture (Block Diagram Explanation)

**Key Components:**

- **ALU:** Performs arithmetic and logic operations
- **Accumulator (A):** Primary register used in operations
- **B Register:** Used in multiplication and division
- **Program Counter (PC):** Holds address of next instruction
- **Data Pointer (DPTR):** Used for external memory access
- **Stack Pointer (SP):** Points to top of stack (default = 07H)
- **Ports (P0 to P3):** 8-bit bidirectional I/O ports
- **ROM/EPROM:** 4KB on-chip program memory
- **RAM:** 128B internal data memory
- **Timers/Counters:** Timer 0 & Timer 1 (16-bit)
- **Serial Port:** Full duplex communication via SBUF
- **Interrupt Control:** 5 sources including serial, timer, external

## 7. Internal Memory Structure

| Range | Use |
|---|---|
| 00H-1FH | Register banks (Bank 0-3) |
| 20H-2FH | Bit-addressable RAM |
| 30H-7FH | General purpose RAM |
| 80H-FFH | Special Function Registers |

**Note:**

- 4 banks of R0–R7 (bank switching via PSW)
- 128-bit addressable bits in 20H–2FH

## 8. Special Function Registers (SFRs)

| Register | Address | Description |
|---|---|---|
| A | E0H | Accumulator |
| B | F0H | Used with A in MUL/DIV |
| PSW | D0H | Program Status Word (flags + bank) |
| SP | 81H | Stack Pointer |
| DPL/DPH | 82H/83H | Data Pointer |
| P0-P3 | 80H,90H,A0H,B0H | I/O Ports |
| TMOD | 89H | Timer Mode Control |
| TCON | 88H | Timer Control |
| SCON | 98H | Serial Control |
| SBUF | 99H | Serial Data Buffer |
| IE | A8H | Interrupt Enable |
| IP | B8H | Interrupt Priority |

## 9. PSW (Program Status Word)

| Bit | Symbol | Description |
|---|---|---|
| 7 | CY | Carry Flag |
| 6 | AC | Auxiliary Carry |
| 5 | F0 | User-defined flag |
| 4 | RS1 | Register bank select |
| 3 | RS0 | Register bank select |
| 2 | OV | Overflow Flag |
| 1 | — | Reserved |
| 0 | P | Parity Flag |

## 10. Pin Diagram and Description

- **40 pins total**
- Power: Vcc (40), GND (20)
- Oscillator: XTAL1 (19), XTAL2 (18)
- Reset: RST (9)
- I/O Ports: P0 (32-39), P1 (1-8), P2 (21-28), P3 (10-17)
- Control: ALE (30), PSEN (29), EA (31)

**P3 Alternate Functions:**

- P3.0: RXD (Serial In)
- P3.1: TXD (Serial Out)
- P3.2: INT0
- P3.3: INT1
- P3.4: T0 (Timer 0 input)
- P3.5: T1 (Timer 1 input)
- P3.6: WR (Ext RAM write)
- P3.7: RD (Ext RAM read)

## 11. Addressing Modes

| Mode | Example | Description |
|---|---|---|
| Immediate | MOV A, #25H | Data embedded in instruction |
| Direct | MOV A, 30H | Access RAM/SFR using address |
| Register | MOV A, R0 | Operands in R0–R7 |
| Register Indirect | MOV A, @R0 | Address from R0/R1 |
| Indexed | MOVC A, @A+DPTR | Access code memory (lookup table) |

## 12. Summary Keywords for Revision

- **8051 = 40-pin DIP, 8-bit CPU, Harvard**
- **SFRs in 80H–FFH range**
- **RAM = 128B + 128B SFR, ROM = 4KB**
- **Ports: P0–P3, some dual-purpose**
- **Timers, Serial, and External Interrupts**
- **PSW: CAPO → Carry, Aux Carry, Parity, Overflow**

**8051 Microcontroller - Module 2 Detailed Notes**

## 1. Instruction Set Classification

The 8051 instruction set is divided into the following categories:

| Category | Examples | Description |
|---|---|---|
| Data Transfer | MOV, XCH, PUSH, POP, XCHD | Move and exchange data |
| Arithmetic | ADD, ADDC, SUBB, INC, DEC | Perform arithmetic operations |
| Logical | ANL, ORL, XRL, CLR, CPL, RL | Logical and bitwise operations |
| Branching | SJMP, LJMP, AJMP, JC, JNC, CALL | Control flow operations |
| Bit Manipulation | SETB, CLR, JB, JNB, CPL, ANL | Single-bit operations |
| Control | NOP, RET, RETI | Control execution (NOP, subroutines, ISR) |

8051 has 255 instructions, and each takes 1–2 machine cycles.

## 2. Addressing Modes

Addressing mode defines the method of accessing operands in an instruction.

| Addressing Mode | Syntax Example | Description |
|---|---|---|
| Immediate | MOV A, #0AH | Data provided in the instruction |
| Direct | MOV A, 30H | Directly accesses internal RAM or SFR |
| Register | MOV A, R2 | Uses R0 to R7 |
| Register Indirect | MOV A, @R0 | Access data via address in R0/R1 |
| Indexed | MOVC A, @A+DPTR | Access code memory (for look-up tables) |

## 3. Data Transfer Instructions

- **MOV A, #data**: Load immediate data into accumulator
- **MOV Rn, direct**: Move from direct RAM to register
- **PUSH direct / POP direct**: Push/pop value on the stack
- **XCH A, Rn**: Exchange A with Rn
- **MOVX / MOVC**: External and code memory access

## 4. Arithmetic Instructions

- **ADD A, Rn/direct/#data**: Add with accumulator
- **ADDC A, Rn**: Add with carry
- **SUBB A, Rn**: Subtract with borrow
- **INC/DEC A or Rn**: Increment/decrement
- **MUL AB / DIV AB**: Multiply or divide A and B
- Flags affected: CY, OV, AC

## 5. Logical Instructions

- **ANL, ORL, XRL A, data**: Bitwise AND/OR/XOR
- **CLR A / CPL A**: Clear/complement accumulator
- **SWAP A**: Swap nibbles of A
- **RL, RLC / RR, RRC**: Rotate accumulator left/right (with/without carry)

## 6. Branching Instructions

| Type | Instructions | Purpose |
|---|---|---|
| Unconditional | SJMP, LJMP, AJMP | Direct jump |
| Conditional | JC, JNC, JZ, JNZ | Based on flag (CY, Z) |
| Call/Return | ACALL, LCALL, RET, RETI | Subroutine and interrupt control |
| Bit-level | JB, JNB, JBC | Branch on bit condition |

- **SJMP**: Short jump (±127 bytes)
- **LJMP**: Long jump (within 64KB)
- **AJMP**: Absolute jump (within 2KB page)

## 7. Bit Instructions

- **SETB bit / CLR bit**: Set/clear bit
- **CPL bit**: Complement bit
- **ANL C, bit / ORL C, bit**: Bitwise AND/OR with carry
- **MOV C, bit**: Copy bit to carry
- **JC, JNC, JB, JNB, JBC**: Conditional bit jump

## 8. PSW Flags Affected by Instructions

| Flag | Instructions Affecting It |
|---|---|
| CY | ADD, ADDC, SUBB, RRC, etc. |
| OV | ADD, ADDC, SUBB, MUL, DIV |
| AC | ADD, ADDC, SUBB |
| P | Set/cleared based on result parity |

## 9. Stack and Stack Pointer

- **Stack**: Temporary storage (LIFO) in internal RAM
- **SP**: Stack pointer (default = 07H)
- **PUSH**: SP++; store data at [SP]
- **POP**: load data from [SP]; SP--

Example:

```
MOV SP, #40H
MOV A, #55H
PUSH ACC  ; SP = 41H, 55H stored
POP B     ; B = 55H, SP = 40H
```

## 10. Assembler Directives

| Directive | Description | Example |
|---|---|---|
| ORG | Set code start address | ORG 0000H |
| DB | Define byte (constants, strings) | MESSAGE DB 'HELLO',0 |
| EQU | Define constant without allocating memory | SIZE EQU 10 |
| END | End of source file | END |
| CODE/DATA | Declare code/data memory segment | SEGMENT CODE |

## 11. Sample Programs

### a. Toggle P1.0:

```
START:
    CPL P1.0
    ACALL DELAY
    SJMP START
```

### b. Add two numbers:

```
MOV A, #05H
ADD A, #03H  ; A = 08H
```

### c. Move block of data:

```
MOV R0, #30H
MOV R1, #40H
MOV A, @R0
MOV @R1, A
```

## 12. Summary Keywords

- **5 instruction categories**: Data, Arithmetic, Logic, Branch, Bit
- **Addressing modes**: I, D, R, RI, IX
- **Directives**: ORG, DB, EQU, END
- **Stack starts at 08H onward**
- **Flags affected in PSW**: CY, OV, AC, P

**8051 Microcontroller - Module 3 Detailed Notes**

## 1. Memory Organization of 8051

### Internal Memory:

- Total = 256 Bytes (divided into:
    - **Lower 128 Bytes (00H–7FH)**: General-purpose RAM + Bit-addressable area
    - **Upper 128 Bytes (80H–FFH)**: Special Function Registers (SFRs)

| Address Range | Description |
|---|---|
| 00H–1FH | Register Banks (4 × 8 = 32 bytes) |
| 20H–2FH | Bit-addressable memory (128 bits) |
| 30H–7FH | General-purpose RAM |
| 80H–FFH | SFRs (Ports, Timers, Serial, etc.) |

### Register Banks:

- 4 Banks: Bank 0 (00H), Bank 1 (08H), Bank 2 (10H), Bank 3 (18H)
- Each has 8 registers: R0–R7
- Selected using **PSW (RS0, RS1)** bits

**External Memory:**

- Up to **64KB external code memory** (ROM)
- Up to **64KB external data memory** (RAM)
- Accessed via **MOVX**, **MOVC** instructions

## 2. I/O Port Programming

**Ports:**

- 4 Parallel 8-bit I/O Ports: **P0, P1, P2, P3**

| Port | Function | Special Use |
|---|---|---|
| P0 | Dual purpose | AD0–AD7 (Address/Data lines) |
| P1 | I/O only | No alternate function |
| P2 | Dual purpose | High-order address (A8–A15) |
| P3 | I/O + control | INT0, INT1, TxD, RxD, WR, RD, T0, T1 |

**Output Example:**

```
MOV P1, #0FFH ; Set all P1 pins HIGH
```

**Bit Addressing Example:**

```
SETB P1.0     ; Set bit P1.0 (LED ON)
CLR P1.0      ; Clear bit P1.0 (LED OFF)
```

**Toggle Port:**

```
CPL P2.3      ; Complement/toggle bit 3 of Port 2
```

## 3. Timers and Counters in 8051

**Overview:**

- Two 16-bit timers: **Timer 0 (T0)** and **Timer 1 (T1)**
- Can work as:
  - Timer: Increments with internal clock
  - Counter: Increments with external pulses (T0 = P3.4, T1 = P3.5)

**Timer Control Registers:**

- **TMOD (Timer Mode Control Register):**
  - 8-bit register: lower 4 bits for Timer 0, upper 4 bits for Timer 1
  - Format:

TMOD = | GATE | C/T | M1 | M0 |
| 1 | 0 | 1 | 0 | → Mode 2 (8-bit auto-reload)
```
```

| Bits | Description |
|---|---|
| GATE | 1 = Start via external pin INTx |
| C/T | 0 = Timer, 1 = Counter |
| M1:M0 | Timer mode selection (00 to 11) |

- **TCON (Timer Control Register):**
  - TF1/TF0 = Overflow flags
  - TR1/TR0 = Timer Run control

| Bit | Name | Function |
|---|---|---|
| TR0 | Start/Stop Timer 0 | |
| TF0 | Overflow flag Timer 0 | |
| TR1 | Start/Stop Timer 1 | |
| TF1 | Overflow flag Timer 1 | |

### Timer Modes:

| Mode | Description | Bits |
|---|---|---|
| 0 | 13-bit timer | M1=0, M0=0 |
| 1 | 16-bit timer | M1=0, M0=1 |
| 2 | 8-bit auto-reload | M1=1, M0=0 |
| 3 | Split Timer (T0 only) | M1=1, M0=1 |

## 4. Timer Delay Calculation

**Machine Cycle Time = 12 / Crystal Frequency**

Example: Crystal = 12 MHz → Machine cycle = 1 µs

**Delay = (65536 - THx:TLx) × 1 µs** (for Mode 1)

To generate 1 ms delay:

- Count = 65536 - 1000 = 64536 → THx = FC, TLx = 18

## 5. Counter Mode Operation

- Set **C/T = 1** in TMOD
- Timer increments with **external pulse** at T0 (P3.4) or T1 (P3.5)
- Used for **event counting**

## 6. Common Timer Programs

### a) Generate Square Wave:

```
MAIN:
    CPL P1.0
    ACALL DELAY
    SJMP MAIN

DELAY:
    MOV TMOD, #01H
    MOV TH0, #HIGH
    MOV TL0, #LOW
    SETB TR0
WAIT:
    JNB TF0, WAIT
    CLR TR0
    CLR TF0
    RET
```

### b) Use Timer as Counter:

```
MOV TMOD, #01000000B  ; Counter mode for Timer 1
MOV TH1, #00H
MOV TL1, #00H
SETB TR1
```

## 7. Summary Keywords for Module 3

- **RAM split**: Register banks (00–1F), Bit area (20–2F), GPR (30–7F), SFR (80–FF)
- **Timers**: T0, T1 – 16-bit each
- **Modes**: 0 = 13-bit, 1 = 16-bit, 2 = 8-bit auto-reload, 3 = split
- **TMOD**: Timer mode + control
- **TCON**: Start/overflow control
- **C/T = 1** → Counter

**8051 Microcontroller - Module 4 Detailed Notes**

## 1. Serial Communication in 8051

### Features:

- Full-duplex UART (Universal Asynchronous Receiver Transmitter)
- Uses Timer 1 to set baud rate
- TxD (P3.1) – Transmit, RxD (P3.0) – Receive

### Baud Rate Formula (Mode 1):

```
Baud Rate = (2^SMOD / 32) * (Timer1 Overflow Rate)
```

- For 9600 baud @11.0592MHz: TH1 = 256 - (11.0592 MHz / 38400) = FDH

### Serial Modes (via SCON Register):

| SM0 | SM1 | Mode | Description |
|---|---|---|---|
| 0 | 0 | 0 | 8-bit shift register |
| 0 | 1 | 1 | 8-bit UART, variable baud rate |
| 1 | 0 | 2 | 9-bit UART, fixed baud rate |
| 1 | 1 | 3 | 9-bit UART, variable baud rate |

## 2. SCON, PCON, and SBUF Registers

- **SCON (98H)**: Serial Control Register
  - SM0, SM1: Mode selection
  - REN: Reception enable
  - TB8, RB8: 9th data bit (used in Modes 2/3)
  - TI: Transmit interrupt flag
  - RI: Receive interrupt flag
- **PCON (87H)**:
  - SMOD: Baud rate double bit (bit 7)
  - IDL, PD: Idle/power-down modes
- **SBUF (99H)**:
  - Write to SBUF to transmit
  - Read SBUF to receive

## 3. Serial Programming (C & ASM)

### Basic Transmit Program (C):

```
SCON = 0x50;       // Mode 1, REN enabled
TMOD = 0x20;       // Timer 1, Mode 2
TH1 = 0xFD;        // Baud rate = 9600
TR1 = 1;           // Start Timer

SBUF = 'A';
while (!TI);
TI = 0;
```

### Receive Program:

```
while (!RI);
char data = SBUF;
RI = 0;
```

## 4. Interrupt System in 8051

### Types of Interrupts:

| Interrupt | Vector Addr | Priority Bit | IE Bit |
|---|---|---|---|
| External INT0 | 0003H | PX0 (IP.0) | EX0 |
| Timer 0 | 000BH | PT0 (IP.1) | ET0 |
| External INT1 | 0013H | PX1 (IP.2) | EX1 |

| Interrupt | Vector Addr | Priority Bit | IE Bit |
|---|---|---|---|
| Timer 1 | 001BH | PT1 (IP.3) | ET1 |
| Serial (TI/RI) | 0023H | PS (IP.4) | ES |

### Interrupt Enable Register (IE - A8H):

| Bit | Symbol | Function |
|---|---|---|
| 7 | EA | Enable all interrupts |
| 6 | — | Reserved |
| 5 | ET2 | Timer 2 interrupt (8052) |
| 4 | ES | Enable serial interrupt |
| 3 | ET1 | Enable Timer 1 interrupt |
| 2 | EX1 | Enable External INT1 |
| 1 | ET0 | Enable Timer 0 interrupt |
| 0 | EX0 | Enable External INT0 |

### Interrupt Priority Register (IP - B8H):

- Set bit = high priority
- Clear bit = low priority

## 5. Interrupt Handling Sequence

1. Interrupt occurs
2. Current instruction finishes
3. PC is pushed to stack
4. Control jumps to ISR (vector address)
5. Execute ISR
6. Return via `RETI` (return from interrupt)

## 6. Enabling / Disabling Interrupts

### Enable INT0 and Timer0:

```
SETB EX0
SETB ET0
SETB EA     ; Enable all interrupts
```

### Disable all interrupts:

```
CLR EA
```

## 7. Serial Interrupt Handling (Assembly Example)

```
ORG 0023H
LJMP SERIAL_ISR
```

```
SERIAL_ISR:
    JNB RI, CHECK_TI
    CLR RI
    MOV A, SBUF ; Read received byte
    ; Process A
CHECK_TI:
    JNB TI, EXIT
    CLR TI
    ; Ready to send next byte
EXIT:
    RETI
```

## 8. RS-232 and MAX232 Interfacing

- **RS-232**: Standard for serial communication between DTE and DCE
    - Voltage: +12V for logic 0, -12V for logic 1
- **8051 Logic**: TTL levels (0V = 0, +5V = 1)
- **MAX232 IC**: Converts RS-232 ↔ TTL voltage levels

**Connection:**

| 8051 Pin | MAX232 | RS-232 Connector |
|---|---|---|
| P3.1 (TxD) | T1IN | T1OUT → PC RxD |
| P3.0 (RxD) | R1OUT | R1IN ← PC TxD |

## 9. Summary Keywords for Module 4

- **TxD = P3.1, RxD = P3.0**
- **SCON**: Serial control register
- **PCON**: Baud rate & power control
- **SBUF**: Data register
- **Interrupts**: 5 sources, vector table
- **RS-232 ↔ MAX232 ↔ 8051**
- **RETI**: End of interrupt
- **TI & RI**: Flags for transmit/receive

**8051 Microcontroller - Module 5 Detailed Notes**

## 1. Overview: Device Interfacing and Advanced Interrupt Concepts

Module 5 focuses on:

- Interfacing external devices (LCD, DAC, Stepper Motor)
- Writing programs to control peripherals
- Using interrupts effectively

## 2. LCD Interfacing with 8051

**LCD Basics:**

- Common LCD: **16x2 alphanumeric display**
- Has: RS, RW, EN, D0–D7 pins
- Control via 8-bit or 4-bit mode

**Pin Descriptions:**

| Pin | Name | Description |
|---|---|---|
| 1 | VSS | GND |
| 2 | VDD | +5V supply |
| 3 | VEE | Contrast control |
| 4 | RS | Register Select: 0 = Cmd, 1 = Data |
| 5 | RW | Read/Write: 0 = Write, 1 = Read |
| 6 | EN | Enable (Latch data) |
| 7-14 | D0–D7 | Data Lines |

### Common LCD Commands:

| Code | Function |
|---|---|
| 38H | 2-line, 8-bit mode |
| 0CH | Display ON, cursor OFF |
| 06H | Auto-increment cursor |
| 01H | Clear Display |

### Interfacing Example:

- P1 → D0–D7
- P2.0 → RS, P2.1 → RW, P2.2 → EN

## 3. DAC Interfacing with 8051

### DAC0808:

- 8-bit Digital to Analog Converter
- Converts digital value (0–255) to analog voltage

### Connections:

| DAC0808 Pin | Connected To |
|---|---|
| D0–D7 | Port 1 (P1.0–P1.7) |
| VREF | Reference voltage |
| IOUT | Output current → Op-amp → Voltage |

### Use Case:

- Generate **ramp**, **sine**, or **triangular waveforms**

**Sample C Code:**

```
unsigned char i;
while(1) {
    for(i = 0; i < 255; i++) {
        P1 = i;
        delay();
    }
}
```

## 4. Stepper Motor Interfacing

**Stepper Motor Basics:**

- Moves in fixed steps (e.g., 1.8° per step)
- Requires a **driver (ULN2003)**

**Interface:**

| 8051 Pin | Driver | Motor Coil |
|---|---|---|
| P1.0–P1.3 | IN1–IN4 | A, B, C, D |

**Clockwise Sequence:**

```
0x09 → 0x0C → 0x06 → 0x03
```

**Counter-clockwise:**

```
0x03 → 0x06 → 0x0C → 0x09
```

**Program (CW Rotation):**

```
unsigned char steps[] = {0x09, 0x0C, 0x06, 0x03};
int i;
while(1) {
    for(i = 0; i < 4; i++) {
        P1 = steps[i];
        delay();
    }
}
```

## 5. Using Switch Input to Control Devices

Example: Switch on P2.7 to choose direction

```
if (P2 & 0x80)
    rotate_CCW();
else
    rotate_CW();
```

## 6. Interrupt Priority and Vector Addresses

**IP Register (B8H):**

- Assigns priority to each interrupt
- 1 = High priority

**Default Priority Order:**

1. INT0
2. Timer 0
3. INT1
4. Timer 1
5. Serial

**Interrupt Vector Table:**

| Interrupt | Vector Addr |
|---|---|
| INT0 | 0003H |
| Timer 0 | 000BH |
| INT1 | 0013H |
| Timer 1 | 001BH |
| Serial | 0023H |

## 7. Applications & Use Cases

- **Stepper Motor**: Precise positioning (printers, robotics)
- **LCD**: Display data, status, values
- **DAC**: Analog signal output, waveform generation
- **Interrupts**: Efficient CPU usage in event-driven systems

## 8. Summary Keywords

- **LCD RS, RW, EN**: Control pins
- **DAC0808**: Digital to analog converter
- **Stepper**: ULN2003 driver, 4-step sequence
- **P2.7**: Switch input for control
- **Interrupt priority**: Configurable via IP
- **Waveform generation**: DAC + looped data

## Mind Map: 8051 Microcontroller – Module 1

```
                  +----------------------+
                  |    8051 MODULE 1     |
                  +----------------------+
                            |
     +----------------------+------------------------+
     |                      |                        |
[1]  Microcontroller     [2] Architecture        [3] Memory & I/O
     Basics                   & Features              Structure
     |                      |                        |
     ├─ Defn (CPU + RAM)    ├─ 8-bit CPU             ├─ 128B Internal RAM
     ├─ Vs Microprocessor    ├─ 4 I/O Ports (P0-P3)   ├─ 4KB ROM
     ├─ RISC vs CISC         ├─ Timers T0/T1          ├─ Bit/Byte Addr RAM
     └─ Harvard vs Von       └─ Serial UART           └─ SFR area (80H-FFH)

     +----------------------+------------------------+
     |                      |
[4]  Pin Description     [5] Addressing Modes
     & Registers              (Instruction Access)
     |                      |
     ├─ 40 Pins             ├─ Immediate:  `MOV A, #30H`
     ├─ Ports P0-P3         ├─ Direct:     `MOV A, 30H`
     ├─ Special Pins:       ├─ Register:   `MOV A, R0`
     │   ALE, PSEN, EA      ├─ Reg Indirect: `MOV A, @R0`
     ├─ SFRs: A, B, PSW     └─ Indexed:    `MOVC A, @A+DPTR`
     └─ PSW Flags:
        CY, AC, OV, P
```

### 📌 Tips for Memory:

- **RISC vs CISC** → *RISC = Simple & Fast*, *CISC = Complex & Powerful*
- **Harvard = Separate Code/Data**, **Von Neumann = Shared**
- **PSW = Program Status Word** → Remember **CAPO** for CY, AC, P, OV
- **P0** = multiplexed AD0–AD7 → needs pull-up
- **4 Addressing Modes** → I D R I (**Immediate, Direct, Register, Indirect**)

## Mind Map: 8051 Microcontroller – Module 2

```
                    +---------------------------+
                    |      8051 MODULE 2        |
                    +---------------------------+
                                |
        +---------------------+---------------------+
        |                     |                     |
[1] Instruction Set     [2] Addressing Modes    [3] Instruction Types
        |                     |                     |
        ├─ 5 Groups:          ├─ Immediate          ├─ Data Transfer
        |   - Data Transfer   |   MOV A, #55H       |   MOV, XCH, PUSH
        |   - Arithmetic      ├─ Direct             ├─ Arithmetic
        |   - Logical         |   MOV A, 30H        |   ADD, SUBB, INC
        |   - Branching       ├─ Register           ├─ Logical
        |   - Bit Instructions|  MOV A, R1          |   ANL, ORL, CPL
        ├─ Machine Cycles     ├─ Register Indirect  ├─ Control/Branching
        |   (1, 2, or 3)      |   MOV A, @R0        |   SJMP, LCALL, RET
        └─ Flags Affected     └─ Indexed            └─ Bit Instructions
            CY, AC, OV, P       MOVC A, @A+DPTR         SETB, CLR, JB, JNB

        +---------------------+---------------------+
        |                     |
[4] ALP Programming     [5] Assembler Directives
        |                     |
        ├─ Start: ORG 0000H   ├─ ORG: Start address
        ├─ END: End of code   ├─ DB: Define byte
        ├─ Use Labels         ├─ EQU: Define constant
        ├─ Use Comments       ├─ CODE / DATA segment
        └─ Delay Loops        └─ END: End of file

        +---------------------+
        |
[6] Program Examples
        |
        ├─ Add two numbers: `MOV A, #05H` + `ADD A, #03H`
        ├─ Data copy: `MOV R0, #30H` → `MOV A, @R0`
        ├─ Bit toggle: `CPL P1.0`
        ├─ Square wave on pin
        └─ Count 1s/0s in a byte (loop + RRC)
```

## ➡ Quick Memory Cues

| Topic | Tip / Mnemonic |
|---|---|
| Addressing Modes | **I D R I X** → Immediate, Direct, Register, Indirect, IndeXed |
| Data Types | 8-bit only in 8051 |
| PSW Flags | **CAPO** → Carry, Aux Carry, Parity, Overflow |
| Logical Ops | Think of **bit-level manipulation**: AND, OR, XOR, CPL |
| Branching Types | **SJMP, LJMP** (short/long), **CALL/RET** for subroutines |

## Common ALP Tasks to Practice:

- Toggling LEDs
- Delay generation using loops/timers
- Copying array elements
- Summing two memory locations
- Bit set/reset and checking via `JB`, `JNB`

## 8051 Microcontroller – Module 3 Mind Map

```
                        +----------------------------+
                        |     8051 MODULE 3          |
                        +----------------------------+
                                    |
   +------------------------------+-----------------------------+
   |                              |                             |
[1] Memory Organization     [2] I/O Port Programming       [3] Timers & Counters
   |                              |                             |
   ├─ 128B Internal RAM           ├─ Ports: P0-P3                ├─ Timer 0, Timer 1
   │   ├─ 00H-1FH: Register banks│   ├─ P0: AD0-AD7              ├─ Mode 0: 13-bit
   │   ├─ 20H-2FH: Bit Addressable│   ├─ P1-P3: I/O only         ├─ Mode 1: 16-bit
   │   └─ 30H-7FH: GPR            │   └─ Input/Output coding     ├─ Mode 2: 8-bit Auto reload
   ├─ 128B SFR (80H-FFH)          ├─ Bit toggle: CPL P1.0        ├─ Mode 3: Timer 0 split
   └─ External RAM (MOVX)         └─ Port latch behavior         ├─ Timer vs Counter
                                                                 └─ TMOD & TCON Registers

   +------------------------------+-----------------------------+
   |                              |
[4] Timer Programming       [5] Delay Generation & Square Wave
   |                              |
   ├─ TMOD = #01/02/10 etc.       ├─ Use loop for delay
   ├─ TR0/TR1: Start Timer        ├─ Software delay (DJNZ loops)
   ├─ TF0/TF1: Overflow Flags     ├─ Timer delay: Load TH/TL
   └─ Count pulses: C/T bit       ├─ Square wave via toggle + delay
                                  └─ Example: 10kHz or 3ms/7ms wave

   +------------------------------+
   |
[6] Programs & Applications
   |
   ├─ Square wave generation using timer
   ├─ Delay creation using Mode 1
   ├─ External pulse counting (C/T=1)
   ├─ Displaying counter value on Port
   └─ Timer-controlled LED or buzzer
```

### Quick Memory Triggers

| Concept | Shortcut |
|---|---|
| **TMOD bits** | GATE |
| **Timer Modes** | M1 M0: 00=13-bit, 01=16-bit, 10=8-bit auto |
| **Start Timer** | `SETB TR0` or `TR1` |
| **Stop Timer** | `CLR TR0` or `TR1` |
| **Flags** | TF0/TF1 for overflow, cleared in software |
| **Counting External** | `C/T = 1`, input on T0/T1 (P3.4/P3.5) |

### □ Common Programs in Exams (Write These)

- Delay using Timer 0 in Mode 1
- Square wave on P1.4 using Timer 0 in Mode 2
- External counter using T1 (P3.5), display on P1
- Timer overflow interrupt (advanced)

.

## 8051 Microcontroller – Module 4 Mind Map

```
                        +-------------------------------+
                        |      8051 MODULE 4            |
                        +-------------------------------+
                                    |
      +---------------------------+---------------------------+
      |                           |                           |
[1] Serial Communication     [2] Interrupt Structure     [3] Special Registers
      |                           |                           |
      ├─ UART (Full Duplex)       ├─ INT0, INT1               ├─ SCON: Serial Control
      ├─ Modes (SCON):            ├─ Timer0, Timer1           │   ├─ SM0/SM1: Mode
      │   - Mode 1 (8-bit, used)  ├─ Serial Tx/Rx             │   ├─ REN: Enable receive
      ├─ Start/Stop Bits          ├─ IE: Enable interrupts    │   ├─ TI/RI: Flags
      ├─ Baud rate (Timer 1)      ├─ IP: Set priorities       ├─ IE: Interrupt Enable
      └─ TH1 = 256 - (Freq/384×Baud) └─ Vector table, RETI    ├─ IP: Interrupt Priority

      +---------------------------+---------------------------+
      |                           |
[4] RS-232 & MAX232         [5] Serial Programming & Use Cases
      |                           |
      ├─ RS-232 Voltage: ±12V     ├─ Send: `SBUF = 'A'`
      ├─ TTL ↔ RS232 via MAX232   ├─ Wait: `while(TI==0); TI=0;`
      ├─ P3.0 = RxD, P3.1 = TxD   ├─ Receive: `while(RI==0); A=SBUF; RI=0;`
      └─ Handshaking: RTS/CTS     ├─ Continuous transmission
                                  └─ Echo/Compare programs

      +---------------------------+
      |
[6] Interrupt Programming
      |
      ├─ ISR Syntax: `ORG Addr → LJMP ISR_NAME`
      ├─ RETI: Return from ISR
      ├─ Enable: `SETB EA`, `SETB EX0`, etc.
      ├─ Priority: Set IP bits
      └─ External, Timer, Serial ISRs
```

## Quick Keywords for Revision

| Concept | Keyword/Value |
|---|---|
| Baud Rate | TH1 = 256 - (Freq ÷ 384 × Baud) |
| SCON | SM0, SM1, REN, TI, RI |
| IE Register | EA, EX0, ET0, EX1, ET1, ES |
| IP Register | PX0, PT0, PX1, PT1, PS |
| INT0, INT1 Pins | P3.2 and P3.3 |
| Serial Ports | Tx = P3.1, Rx = P3.0 |
| MAX232 | TTL ↔ RS-232 converter |
| RETI | Return from ISR |

## Common Programs to Practice:

- Transmit “HELLO” via serial port @9600 baud
- Interrupt-driven serial receive and echo
- Timer interrupt to blink LED
- External interrupt (e.g., switch on INT0 triggers buzzer)

## 8051 Microcontroller – Module 5 Mind Map

```
                        +-----------------------------+
                        |      8051 MODULE 5          |
                        +-----------------------------+
                                     |
     +-----------------------------+-----------------------------+
     |                             |                             |
[1] Interrupt System         [2] External Interfacing       [3] Device Drivers
     |                             |                             |
     ├─ INT0 (P3.2), INT1 (P3.3)   ├─ LCD (16×2): RS, RW, EN      ├─ DAC 0808 (analog out)
     ├─ Timer Interrupts           ├─ DAC: Digital → Analog       ├─ Stepper motor control
     ├─ Serial Interrupt           ├─ Stepper motor: ULN2003      ├─ Square wave generation
     ├─ Vector Table               ├─ Sensor or Keypad            └─ Conditional control
(switch)
     ├─ Enable (IE) / Priority (IP)|
     └─ `RETI` in ISR              └─ Port-level interfacing

     +-----------------------------+-----------------------------+
     |                             |
[4] LCD Interfacing          [5] Stepper Motor Interfacing
     |                             |
     ├─ 8-bit or 4-bit mode        ├─ Clockwise / Anticlockwise
     ├─ P1 = Data, P2.0-P2.2 = Ctrl├─ 4-step/8-step sequence
     ├─ Commands: 38H, 0CH, 06H    ├─ ULN2003 as driver
     ├─ Display "IOT", "HELLO"     ├─ 90 steps = 4°/step
     └─ Delay needed for enable    └─ Switch direction via P2.7

     +-----------------------------+
     |
[6] DAC Programming
     |
     ├─ DAC0808: D0-D7 inputs
     ├─ Port (e.g., P1) to DAC pins
     ├─ Convert binary to analog ramp
     └─ Generate sine/ramp/triangular wave
```

### Quick Memory Aids

| Topic | Tip/Keyword |
|---|---|
| External Interrupts | INT0 (P3.2), INT1 (P3.3) |
| DAC | Digital to Analog → use `P1 = A` |
| LCD Commands | `38H` (8-bit), `0CH` (display on), `01H` (clear) |
| Stepper Motor | CW = 0x01, 0x02, 0x04, 0x08 |
| Switch Logic | `if (P2 & 0x80)` → check P2.7 |
| ISR Return | Always use `RETI` |
| LCD Timing | Add delay after `EN` pulse |

### Important Programs to Practice

- Display text on LCD
- DAC ramp wave generation
- Rotate motor CW or CCW using switch input
- Serial interrupt with message receive/send
- External interrupt-based event trigger